Information and Software Technology 52 (2010) 249–273

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

Model-driven development for early aspects

Pablo Sánchez [a,*], Ana Moreira [b], Lidia Fuentes [a], João Araújo [b], José Magno [b,c]

[a] Dpto. Lenguajes y Ciencias de la Computación ETSI Informática, Universidad de Málaga Málaga, Spain
[b] CITI/Departamento de Informática Faculdade de Ciências e Tecnologia, Universidade Nova de Lisboa Lisboa, Portugal
[c] Dpto. Engenharia Informática, Escola Superior de Tecnologia e Gestão, Instituto Politécnico de Leiria Leiria, Portugal

ARTICLE INFO

Article history:
Received 11 September 2008
Received in revised form 28 July 2009
Accepted 22 September 2009
Available online 5 November 2009

Keywords:
Early aspects
Model-driven development
Aspect-Oriented Software Development
Model transformation

ABSTRACT

Currently, non-functional requirements (NFRs) consume a considerable part of the software development effort. The good news is that most of them appear time and again during system development and, luckily, their solutions can be often described as a pattern independently from any specific application or domain. A proof of this are the current application servers and middleware platforms that can provide configurable prebuilt services for managing some of these crosscutting concerns, or *aspects*. Nevertheless, these reusable pattern solutions presents two shortcomings, among others: (1) they need to be applied manually; and (2) most of these pattern solutions do not use aspect-orientation, and, since NFRs are often crosscutting concerns, this leads to scattered and tangled representations of these concerns. Our approach aims to overcome these limitations by: (1) using model-driven techniques to reduce the development effort associated to systematically apply reusable solutions for satisfying NFRs; and (2) using aspect-orientation to improve the modularization of these crosscutting concerns. Regarding the first contribution, since the portion of a system related to NFRs is usually significant, the reduction on the development effort associated to these NFRs is also significant. Regarding the second contribution, the use aspect-orientation improves maintenance and evolution of the non-functional requirements that are managed as aspects. An additional contribution of our work is to define a mapping and transition from aspectual requirements to aspect-oriented software architectures, which, in turn, contributes to improve the general issue of systematically relating requirements to architecture. Our approach is illustrated by applying it to a Toll Gate case study.

1. Introduction

A *software architecture* needs to satisfy the set of functional and non-functional requirements identified during requirements analysis. It is the software architect's responsibility to design an architecture for the system that meets the functional requirements and satisfies the system's quality attributes (or non-functional requirements). Quality attributes such as robustness, security, availability, performance and distribution have a lasting impact on a software architecture, and have acquired a key importance in nowadays software systems. So, protection of personal data, for example, is now so important as any functional requirements of the software system under development.

An important property of these non-functional requirements is that they can be described independently of the system where they are incorporated, by making some assumptions on an abstract system. Thus, we can describe the security requirements and one or more solutions satisfying these requirements, by simply assuming that there are some data in a certain system that must be secured. As a consequence, several of these non-functional requirements have been studied in isolation and different pattern solutions have been proposed. The work of Juristo et al. [59] on usability is an exemplar of this argumentation, as well as the well-known book by Chung et al. [26]. Another important property of these non-functional requirements is that they are recurrent across many different applications. So the pattern solutions for satisfying them can be reused across many different applications and domains. This makes the effort dedicated to study and find pattern solutions for non-functional requirements become cost-effective as these patterns can be reused many times in several different contexts.

Most of these non-functional requirements are identified first at the requirements level and each one may affect several architectural design artefacts. These non-functional requirements are very often *crosscutting*. That is, their representation cannot be encapsulated in a single module and their behaviour is, therefore, scattered and tangled with the core functionalities of other architectural

* Corresponding author. Tel.: +34 952132846; fax: +34 952131397.
E-mail addresses: pablo@lcc.uma.es (P. Sánchez), amm@di.fct.unl.pt (A. Moreira), lff@lcc.uma.es (L. Fuentes), ja@di.fct.unl.pt (J. Araújo), magno@estg.ipleiria.pt (J. Magno).

0950-5849/$ - see front matter © 2009 Elsevier B.V. All rights reserved.
doi:10.1016/j.infsof.2009.09.001

components they have an impact on. Classical software development methods, such as object-oriented or component-based ones, cannot modularize crosscutting concerns. The tyranny of the dominant decomposition is acknowledged as the main cause that hinders an effective separation of concerns, which makes software architecture definition, maintenance, evolution and adaptation to new requirements more difficult [100].

Aspect-Oriented Software Development (AOSD)[1] is an emerging discipline that promotes the separation of crosscutting concerns, by encapsulating them in special modules, the *aspects*. Initial research on AOSD focused mainly on the implementation level. In the last few years, the AOSD concepts have moved beyond programming and have been applied at the earlier software development stages. The term *Early Aspects*[2] focuses on the identification and separation of *crosscutting concerns* at the requirements and architecture levels. Several Early Aspects approaches[3] have been proposed recently, including Aspect-Oriented Requirements Engineering (AORE) approaches as well as Aspect-Oriented Architecture Design (AOAD) approaches. They all seek to improve the modular representation of crosscutting concerns at the requirements (named *aspectual requirements*) and the architecture level (named *architectural aspects*).

AORE approaches encapsulate crosscutting concerns in separate modules and special composition rules are defined to support their influence on each other, as well as to identify trade-offs among them. Therefore, AORE approaches contribute to improve the management of non-functional requirements [87,106,79,18,107]. The application architecture should be designed bearing in mind the results of such a requirement analysis. Initial research has been done to overcome the above mentioned mapping problem. The goal of such work was to map proposals from different authors [6,23,55] providing a set of guidelines that relate aspectual requirements with architectural aspects. However, even when a mapping process is available, a manual application of the process can be a repetitive, laborious and error-prone task.

This paper presents an approach where the mapping of recurrent aspectual requirements, which are often non-functional requirements, into an architectural solution that satisfies such requirements, is automated by means of automatic model transformations. The advantages of our approach are twofold:

(1) Architectural solutions that satisfy non-functional requirements are now better modularised, due to the use of aspect-oriented techniques. This improvement on modularization leads to an improvement on maintenance and evolution, such as pointed out in several empirical studies [45,105,71,49,92,35,36,30,78].
(2) The manual mapping for applying pattern solutions satisfying non-functional requirements is now automated by using model-driven techniques. This avoids that repetitive, laborious and error-prone tasks need to be performed manually, reducing the development effort, and, consequently also increasing quality – as we are eliminating errors introduced by a manual application of a pattern solution. Collateral benefits of the automation of the mapping of non-functional requirements are:
    (a) Transformations from requirements analysis models to architectural models are performed in a consistent manner, as they are automated;
    (b) Best practices and recurring scenarios can be encapsulated in automatic transformations; and
    (c) New requirements can be incorporated into the AORE model and be consistently and automatically propagated to the architecture level.

These ideas are illustrated by applying our approach to a Toll Gate system case study [87], and evaluated by means of applying our approach to other two case studies, an Online Book Store system [76] and an Auction System [94].

In summary, the main contributions of this work can be briefed here, and each one is then extensively discussed in the main body of the paper. Since a significant portion of developing a system is consumed with handling non-functional requirements, one contribution of our work is to reduce the development effort. We have been observing that certain non-functional requirements appear recurrently during system development. Fortunately, their solutions can be described as a pattern, independently from any specific application or domain. The approach presented in this paper contributes to achieve this first goal by using model-driven techniques to reduce the development effort associated to apply reusable solutions for satisfying NFRs and using aspect-orientation to improve the modularization of these crosscutting concerns. Another contribution is related with the increasing importance we have witnessed lately regarding maintenance and evolution as fundamental quality factors for software engineering. Our approach improves maintenance and evolution of the non-functional requirements that are managed as aspects. Finally, a significant contribution is also the definition of a clear mapping and transition from aspectual requirements to aspect-oriented software architectures, which contributes to improve the general issue of systematically relating requirements to architecture. In this paper, we establish clear links between the requirements-level aspects and the architectural-level aspects.

The remaining of this paper is structured as follows: Section 2 provides a background on aspect-orientation. Section 3 introduces model-driven development, highlighting some of its main characteristics. Section 4 gives a general description of our approach. Section 5 illustrates the approach we developed by applying it to a case study. Section 6 comments on supporting tools. Section 7 contains an evaluation and a critical discussion on the work presented. Section 8 discusses some related work. Finally, Section 9 summarises the paper and gives directions for future work.

2. Aspect-oriented software development

Software development is a discipline in constant evolution. Its main focus has always been on improving abstraction, modularisation and composition techniques [88]. In recent decades, several software development paradigms, such as Object-Oriented and Component-Based development, have been proposed with this principle in mind. While these paradigms represent a large step forward with respect to abstraction, modularity and composability, there are still some concerns that do not align well with their decomposition criteria and are, therefore, difficult to modularise. The final result is that the specification and implementation of those concerns are typically scattered across several core (base) modules, producing tangled representations that are difficult to maintain, reuse and evolve. Such properties are known as *crosscutting concerns*, of which, logging, security and persistence are classical examples in the literature [62].

This section starts by introducing some fundamental concepts of aspect-orientation. Then follows by giving an overview of AORA (Aspect-Oriented Requirements Analysis), the aspect-oriented requirements approach we will use to create the aspects-oriented specification, and finishes by summarizing CAM (Component

[1] http://www.aosd.net.
[2] http://www.early-aspects.net.
[3] The interested reader can find a survey of such approaches in [23].

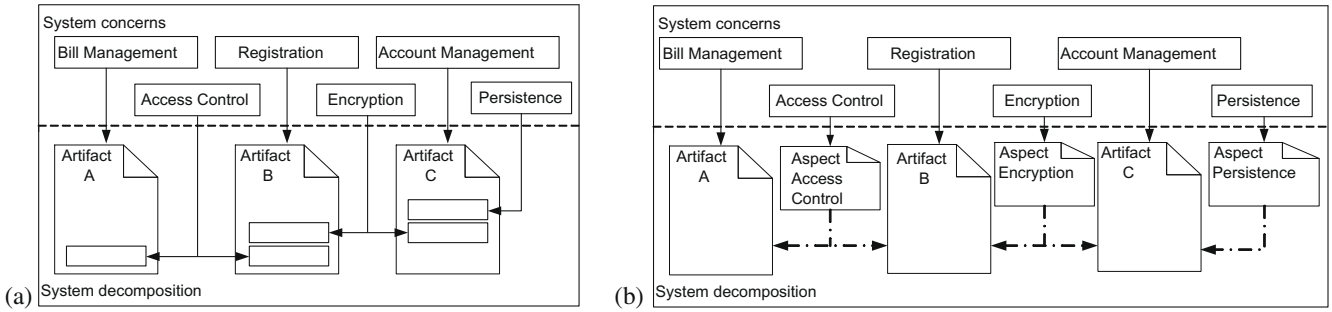

Fig. 1. Classical software decomposition (left); modularisation of crosscutting concerns (right).

Aspect Model), the aspect-oriented architectural design language used in this article.

2.1. Overview on aspect-oriented concepts

Aspect-Oriented Software Development (AOSD)[4] aims at providing improved modularisation and composition techniques to handle crosscutting concerns. Crosscutting concerns are encapsulated in separate modules, known as *aspects*, and composition, or weaving, mechanisms are later used to weave them back to the base modules in compilation, loading or execution time [9].

Aspect-orientation has its roots in the work on composition filters [14] adaptive programming [74], subject-oriented programming [51] or the meta-object protocol [61]. Since then, many aspect-oriented programming languages have appeared, such as Apostole [4], an extension to Smalltalk, AspectC++ [47], an extension to C++, and Aspect# [67], an extension to C#. But new languages were also created, like CaesarJ [2] and AspectWerkz [16]. From all these, AspectJ [63] is the best known, in our opinion.

An interesting phenomenon in Software Development is that the new concepts are usually first handled at the implementation level and later tend to be abstracted and propagated to the earlier phases of the software lifecycle [88]. We witnessed this evolution tendency before with both structured and object-oriented programming, for example. A similar trend has also been happening to aspect-orientation. Aspect-oriented concepts were first experimented at the implementation level and have later moved up the software development lifecycle, to software design [28,38,52,55,99,41], software architecture [11,13,60,85] and requirements engineering [8,20,79,87,7,18].

The problems caused by the difficulties in modularising crosscutting concerns are illustrated in Fig. 1(left). The concerns participating in a specific system are shown in the upper level and the system decomposition artefacts for a particular stage (requirements, architecture, implementation, etc.) are shown below the dotted line. While some concerns are well modularised in specific artefacts, for example `BillManagement`, `Registration` and `AccountManagement`, others are scattered in some of these artefacts. For instance, whereas `AccessControl` is scattered among artefacts `A` and `B`, artefact `B`, which should ideally only encapsulate `Registration`, is tangled with concerns `AccessControl` and `Encryption`. Therefore, crosscutting concerns hinder software maintainability and evolution [27].

AOSD modularises crosscutting concerns in special modules, the aspects. Fig. 1(right) illustrates this idea, where special composition rules, represented as dotted-dashed lines, indicate how the aspects (`AccessControl`, `Encryption` and `Persistence`) are woven into the base units (artefacts `A`, `B` and `C`). The composition rules promote aspects reuse, offering a separate mechanism to specify how and when crosscutting concerns affect the base modules.

When applied to requirements engineering, base artefacts can be use cases [54], viewpoints [37] or goals [73], for example. Aspect-Oriented Requirements Engineering provides a systematic means for the identification, modularisation, representation and composition of crosscutting properties, both functional and non-functional. These crosscutting concerns are described in separate modules and special composition rules are offered to support influence and trade-off analysis before the architecture design is derived [87,79].

When applied to software architecture, base units are components and aspects are aspectual components which encapsulate crosscutting behaviours. Aspectual components are composed (or woven, in aspect-oriented programming terminology) with base components to obtain the whole application. Following the component technology principles, the points in the base component (*join points*, in aspect-oriented programming terminology), where aspect behaviour can be added, can only be those that appear as part of the component public interfaces, e.g., component creation and destruction or message sending and receiving.

2.2. Some basic ideas on the AORA approach

The Aspect-Oriented Requirements Analysis (AORA) approach was proposed for analysing concerns, focusing specially on crosscutting concerns, or *aspects*, during requirements engineering [18]. AORA focuses on:

- Identification of crosscutting concerns, functional or non-functional.
- Modularization and encapsulation of crosscutting and non-crosscutting concerns.
- Composition of crosscutting and non-crosscutting concerns.
- Identification and resolution of conflicts that could emerge at the Requirements Engineering level during composition.

The AORA model is composed of three main tasks, each one refined into several subtasks:

1. *Identify Concerns.* This task consists of identifying all the concerns of a system, where a concern refers to a property addressing a certain issue that is of interest to one or more stakeholders and that can be defined as a set of coherent requirements. Each concern defines a property that the future system must provide. This task is divided into six subtasks: identify stakeholders, identify sources, elicit concerns, reuse of catalogues, decompose concerns, and classify concerns.
2. *Specify Concerns.* This task is divided into four subtasks: identify responsibilities, identify contributions, identify required concerns, and identify stakeholders' priorities. The

[4] http://www.aosd.net.

contributions help detecting conflicts whenever concerns contribute negatively to each other. The required concerns field is used to identify crosscutting relationships.

3. *Compose Concerns*. This task's goal is to give the developer a view of the whole system and to identify and manage conflicts between concerns that might result from the context-dependent compositions. To guide the composition, we propose four subtasks: identify match points, identify crosscutting concerns, handle conflicts, and define composition rules. AORA supports incremental composition, meaning that composition is allowed between any kind of concern, crosscutting or non-crosscutting.

From the above concepts, *match point* is the one that needs to be discussed in some more detail in this paper. A match point defines which concerns should be composed, where one always plays the role of the base concern. A concern that appears in more than one match point is a crosscutting concern.

Each composition takes place in a match point in the form of a composition rule. A composition rule shows how a set of concerns can be woven together by means of a pre-defined operator. AORA supports four operators, inspired by the following LOTOS operators [17]:

- Enable (T1 $\gg$ T2): sequential composition.
- Disable (T1 [> T2): the behaviour of T1 is substituted by the behaviour of T2.
- Parallel (T1 || T2): the behaviours of T1 and T2 must be synchronized (they are concurrent).
- Choice (T1 [] T2): only one of the concerns will be satisfied (T1 or T2).

The composition rule takes the form <Operand><Operator><Operand>, where the operands can be a concern or a sub-composition (which is another composition rule) and <Operator> represents the operator.

The composition task, apart from allowing the construction of views over a system, identifies and manages conflicting concerns that might result from compositions. Conflicts between aspects are a type of "aspect interaction". Indeed, aspect interaction is a well-known problem inside the aspect-oriented community, but for which no full sound solutions exist [1,32,33,34,56,66,91,108,21]. Most of the existing work focus on identifying dependencies and interactions between aspects (e.g., [1,56,91]), but few ones exist on how to provide solutions for aspect-interaction (e.g., [32]), and when they exist, it is often a solution for a very specific kind of interaction between a particular pair of aspects.

This is exactly the case of what happens in AORA, where we can identify some "aspect-interactions", which are no more than conflicting situations that happen when non-functional requirements contribute negatively to each other, they have to be composed in the same match point. One form of resolution of this type of aspect interactions can be found in [21,18].

At the architectural level, the Component Aspect Model (explained in the next section) also provides some support for managing aspect-interaction. But, currently, we have not encoded anything in the model transformations for dealing with aspect-interaction, as it is still part of the research agenda in aspect-orientation as well as product lines. We aim to solve this as part of our future work.

2.3. The component aspect model (CAM)

There are several aspect-oriented architectural design languages, such as AspectualACME [46], Fractal [83] and PRISMA [82]. In this paper, we have opted to work with the UML 2.0 Profile for CAM [85] because: (1) it is a UML 2.0 Profile, so machine-readable models that conform to a metamodel [68,77] and tool-support can be obtained without any extra effort and (2) we have considerable previous experience in using it.

Using the UML 2.0 Profile for CAM, a software architectural model is organized into two main views:

1. A structural view that specifies its constituent components and interfaces, and how these components and interfaces are interconnected. This structural view is modelled using UML 2.0 component diagrams.
2. A behavioural view that models the interactions between these components. This behavioural view is modelled using UML 2.0 sequence diagrams.

Aspects are considered a special kind of component in CAM. These aspect components are composed with the other components differently from traditional components. How aspect components are triggered based on common component interactions is specified in the behavioural view of the software architecture.

In this profile, components are represented as common UML 2.0 components. Aspects are depicted as a special kind of component, stereotyped as «aspect». Provided and required interfaces are represented in the usual UML 2.0 notation. CAM components never interchange messages directly. Instead, they communicate through their ports. Messages sent to a port from outside a component are forwarded to the component internals, while messages sent to the port from the inside are forwarded to the connected external components. This approach enables the sender to declare required interfaces, and to send messages to its own ports when communicating with the environment, rather than identifying an external target component directly. Components defined in this way are assembled by wiring them together by means of provided and required interfaces.

In the behavioural view, when an aspectual behaviour, i.e., an advice in (an *advice* in AspectJ [63] terminology), must be implicitly triggered by the aspect-oriented weaver on a component interaction is indicated by placing message stereotyped as «aspectual» in a sequence diagram, from a component port to the aspect. This does not mean there is an explicit call from the component to the aspect, or that component is explicitly connected to the component. Rather, this means is that between the time when a port receives a message and the message is dispatched inside or outside, a piece of crosscutting behaviour is executed without the knowledge of the affected component. The call to the crosscutting behaviour is performed implicitly by the aspect-oriented weaver as result of the

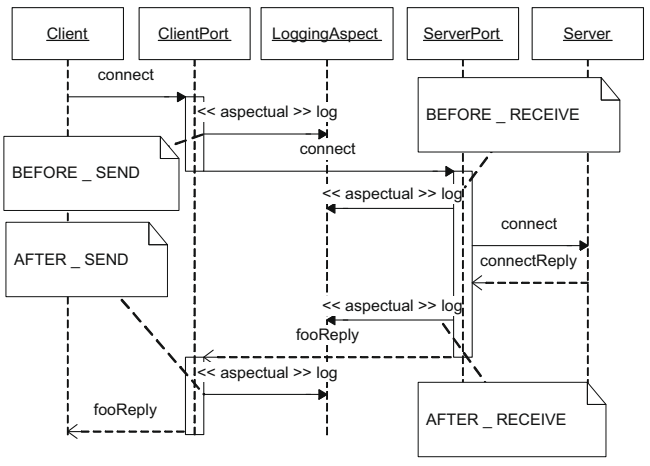

Fig. 2. Aspect execution in the CAM Profile.

weaving process, i.e., the component is oblivious of this call. We would also like to point out that common components are not explicitly connected to aspectual components.

Fig. 2 shows how and when aspects can be executed on a sending/receiving message between components according to the CAM Profile, and how an aspect execution must be modelled concerning the exact point of a sending/receiving message that it crosscuts. We have selected a simple example where a client connects to a server through an invocation to a `connect` method. In order to keep track of connections, a `logging` aspect is used. So, for instance, if we want to specify the log advice must be triggered after the `Server` receive the method call but before it executes it, we would place a call stereotyped as «aspectual» between the point in which `ServerPort` receives the call from outside and the point in which `ServerPort` dispatches this call to the Server component internals. Other interception points, like BEFORE_SEND, AFTER_SEND or AFTER_RECEIVE, are indicated in Fig. 2.

3. Model-driven development

Model-Driven Development (MDD) refers to the systematic use of models and model transformations as primary engineering artefacts throughout the entire software lifecycle. MDD raises the level of abstraction at which developers create software by simplifying and formalising the various activities and tasks that comprise the software development process. The idea is to automate the process of creating new software and to facilitate evolution in a rapidly changing environment by using model transformations [70].

This section addresses the MDD paradigm and gives a brief description of the QVT (Query, View, Transformation) language [86], the Object Management Group (OMG) standard for specifying model transformations.

3.1. Overview on MDD

Model-Driven Development (MDD) is a new technology for Software Development where models are no longer simple mediums for describing software systems or facilitating inter-team communication [68,77,15,81]. Models are now first class citizens of the software development process, and even the code is managed as a model. Using MDD, a software system is obtained through the definition of different models at different abstraction layers. Models of a certain abstraction layer are derived from models of the upper abstraction layer, by means of automatic model transformations.

An automatic model transformation specifies how an output model is constructed based on the elements of an input model. Model transformation languages aim at automating the process of deriving one model from another one. Thus, when the mapping between two different kinds of models is known (e.g., the mapping between an entity-relationship model and a relational database model), model transformations can provide the following benefits [68,77,15]:

- Repetitive, laborious and error-prone tasks, required to create a model from another model are avoided, as transformations are executed by a computer [68,77,15].
- Best practices can be encapsulated in model transformations, ensuring target model quality [93,77,15].
- The mapping process encapsulated in a model transformation can be easily applied, as software developers applying the model transformations do not need to know the details about how the mapping is performed. For example, a database architect can construct an entity-relationship model and then apply a transformation to produce a relational model without knowing how the transformation is exactly performed [68,77,15].
- Changes are less difficult to manage, as they can be done at the corresponding abstraction layer and propagated quickly to lower abstraction levels by model transformations. For example, adopting a replication and load balance strategy in a system can be considered an architectural change. The architectural model in the model-driven process would be updated and then the change propagated to design, implementation and deployment models [68,77,15]. Nevertheless, most of model transformation languages have difficulties to preserve manual changes made to a model when the model is updated, so this kind of round-trip engineering is still an open research issue.
- When several transformations, from a source model to different kinds of target models are available, the same source model can be reused to generate different systems. For instance, if transformations from relational models to SQL and XML models are available, the same relational model can be reused to generate a different implementation of the same database [68,77,15]. This feature is not explored in this paper, as we will consider only one target language. The transformation of an aspect-oriented requirements specification to several aspect-oriented architectural languages, or even non-aspect-oriented ones, has been left as part of our future work.

In this paper, we opted for MDA, the OMG[5] proposal for model transformation [68,77], which enforces the use of a set of standards defined by that organization. The use of standards should help MDA proposals to be more universal, since the compatibility to export and import models to different tools is assured, and a widespread adoption by industry is more probable [68,93].

3.2. The Query, Views and Transformations (QVT) language

QVT is a standard model transformation language, proposed by OMG, which comprises three sublanguages: QVT-Relational, QVT-Operational and QVT-Core. These sublanguages offer different styles and abstraction levels for specifying model transformations.

We have chosen the QVT-Relational language for specifying our automatic model transformations, since it offers a higher abstraction level than the QVT-Operational language and a better visualization of the model transformation specifications. A transformation expressed in QVT-Relational can be viewed as a graph transformation,[6] called *relation*, which contains two patterns based on instances of the metaclasses of the corresponding metamodel and their relationships. Thus, to specify a QVT transformation, in-depth knowledge of the source and target metamodels is required. Transformations are executed in one direction, which determines the source and the target models.

The semantics of a model transformation is the following. Whenever the source pattern is found in the source model, the target pattern must also appear in the target model. To satisfy this relation, it is allowed to create, update and delete objects in/from the target model.

4. Model-driven development for early aspects

This section provides a general overview of our approach. Our aim is to derive an aspect-oriented architecture from an aspect-oriented requirements specification, preserving, whenever

[5] Object Management Group, http://www.omg.org.

[6] Realizing QVT with VMTS (Graph Transformation-Based Model Transformation), http://vmts.aut.bme.hu/qvt/vmts_qvt.html.

possible, the information contained in the requirements specification.

Mapping requirements associated to crosscutting concerns is often based in the systematic application of pre-defined patterns that contains a solution that satisfies the corresponding requirements. For example, for mapping a concern such as *Authenticity* to an architectural (part of) model, there are well-identified patterns in the literature, such as [104], which explain what elements must be introduced in an architectural solution and how these elements must be connected to achieve the desired goals. The application of these *solution patterns for non-functional requirements* requires few or no creativity, as they are often based in the systematic application of a set of steps and operations, which can, in most of cases, be defined an algorithmically, by means of a set of precise and unambiguous steps. It should be noted that when we speak about solution patterns for non-functional requirements, we do not refer to traditional architectural patterns, such as publish-and-subscribe or layered architectures. These patterns require the expertise of the software architect and they cannot be so easily automated.

Thus, the hypothesis for our approach is that these patterns can be encapsulated in model transformations that are automatically applied to requirements models, generating an architectural design model as output. Nevertheless, some guidance is required from the software architect during this process, as we will explain later, in Section 5.4.

Fig. 3 depicts a general overview of the steps that comprise our approach. Each of these steps is described below.

4.1. Step 1: requirements identification

The goal of the *Requirements Identification* (Fig. 3) is to generate an aspect-oriented requirements specification, using any of the currently existing aspect-oriented requirements engineering approaches. This step is only necessary if an AORE specification is not yet available. In situations where an aspectual requirements specification is available, this step may be skipped.

Our approach is independent of the AORE approach selected, and therefore any of the existing aspect-oriented requirements engineering approaches, such as [8,79,87,7,18] can be used for this first step.

The output of this step is a requirements specification describing in detail the system concerns, be them functional, such as `Billing`, or non-functional, such as `Integrity`. Only one constraint is imposed on the output of the process: the system must be decomposed into a set of scenarios, each one containing *only one non-crosscutting sea-level*[7] *functional concern* (a single interaction with the system) and any number of aspectual requirements that have an impact on the functional concern. The reason for this restriction is that these fine-grained scenarios are affected by fewer aspectual requirements, since they contain fewer elements than coarse-grained (kite or sky-level) ones. This means that the development of automatic model transformations and the trade-off analysis at the requirements and architecture levels will be simpler.

Therefore, if appropriate composition mechanisms are provided, larger scenarios can be obtained through consecutive composition of fine-grained ones. For instance, sky and kite-level scenarios might be obtained by composing sea-level ones. In our approach, details of sea-level scenarios are provided in UML packages describing those scenarios. For the composition, just the name of the fine-grained scenario is required. The remaining details

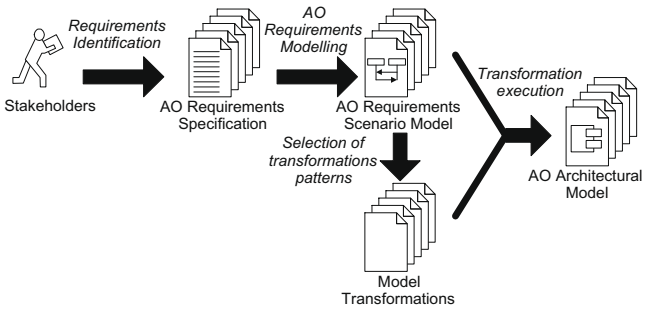

Fig. 3. Process for automatic transformation of an aspect-oriented requirements model into an aspect-oriented architectural model.

about event interactions, for example, are simply abstracted and hidden in the appropriate composition element. For instance, the kite-level scenario `Enter Motorway` in Fig. 7 results from the composition of the four sea-level scenarios `Authorise Vehicle`, `Record Vehicle Entry`, `Show Green Light` and `Report Police`. In this composition, only the names of the fine-grained scenarios are used; the details are hidden in their corresponding sequence diagrams. The kite-level scenario obtained through such composition, as illustrated in Fig. 7, is more manageable than a kite-level scenario where all the interactions were explicitly depicted. Hence, if appropriate composition mechanisms are provided, large scenarios can be obtained easily by means of composing fine-grained ones, where fine-grained requirements are hidden during the composition. This type of composition language provides the required mechanisms to overcome the apparent lack of scalability imposed by our initial restriction of handling only one non-crosscutting sea-level functional concern.

4.2. Step 2: aspect-oriented requirements modelling

A prerequisite for the use of model-driven techniques is the use of machine-readable models (i.e., well-formed models that conform to a metamodel), which a transformation tool can read and process [68,77]. Thus, the second step of our approach is to elaborate such a machine-readable model using the aspect-oriented specification obtained in the previous step.

We start by building UML[8] models based on the obtained aspect-oriented requirements specification. For this purpose, we have developed a UML Profile that permits expressing a requirements specification (the output of the previous step) as a UML 2.0 model. This model serves as input for the automatic model transformation that will generate an aspect-oriented architecture. The use of UML avoids the need to develop dedicated tool-support for building requirements, enabling development teams to use their favourite UML editors. Using this Profile, aspect-oriented requirements are modelled as a set of scenarios, each one containing one non-crosscutting functional concern that may be affected by several aspectual concerns. The output of this process is an aspect-oriented requirements scenario model in UML.

4.3. Step 3: selection of transformation patterns

When mapping a certain concern, specially crosscutting concerns, several solutions may be initially available. For instance, when mapping a crosscutting concern such as response time to an architectural design, the following three alternatives can be valid solutions initially:

[7] According to (Cockburn, 2001), requirements can be described at different levels of granularity: *sky* (broad system goals), *kite* (more detailed goals containing several subgoals), *sea* (a single system interaction), or *mud* (low-level and highly detailed requirements).

[8] Unified Modeling Language (UML), http://www.uml.org/.

(1) the addition of a cache to the system;
(2) a replication and load balance strategy;
(3) a combination of the previous two solutions.

It is the responsibility of the software architect to analyse the different available alternatives and to select the most appropriate one according to the user needs. Each of these solutions is associated to a certain pattern that can be encapsulated into an executable model transformation. This means that the software architect must configure the transformation process by selecting the set of transformations that best satisfies the required quality attributes for the specific system under development.

Since the transformations can be automatically executed, an advantage of our approach is that software architects may want to execute several model transformations to generate several candidate architectures, each one attending different architectural design decisions, before s/he decides on a particular alternative. Each of the resulting architecture candidates can then be analysed and the one that best satisfies the qualities we wish for, selected. This can be achieved by using an aspect-oriented architectural analysis method, such as ASAAM [102].

4.4. Step 4: transformation execution

The aspect-oriented requirements scenario model is automatically transformed into an aspect-oriented architectural model using the pre-defined automatic model transformations selected in the previous step. The aspect-oriented architectural model is generated incrementally by automatically transforming each scenario created in Step 2 individually. In order to transform each scenario, each non-aspectual functional requirement is processed first and, then, an architectural solution that satisfies the aspectual requirement is injected into the generated architecture to ensure that all the information contained in the requirements model is used at the architectural level. The architectural model is expressed using the UML 2.0 Profile developed for CAM [85]. Model transformations are specified using the QVT (Query, View, Transformations) standard [86] and implemented in ATL [58].

Finally, we would like to point out that aspectual requirements are not always transformed into an architectural artefact. Sometimes, according to [87,79,90], aspectual requirements may be mapped to architectural decisions, architectural constraints that might not be possible to express in UML, or they may simply be postponed and not addressed at the architectural level. To properly handle this issue, an auxiliary traceability repository would help to record (a) the reason behind a specific architectural decision, (b) the architectural constraint and the elements it affects and (iii) the reason why a concern cannot be addressed immediately and, instead, is deferred to be handled later. Traceability is, however, beyond the scope of this paper. The interested reader can refer to [90,25].

The goal of this paper is to generate an aspect-oriented software architecture, which is expressed in the UML 2.0 Profile for CAM. So, once we have been able to generate our architecture, the work of this paper is done. Nevertheless, there are several paths that can be followed after generating this aspect-oriented software architecture. It is up to each development team to decide which path should be followed. We enumerate several of these options below:

(1) The CAM aspect-oriented software architecture can serve as basis for a manual detailed design or implementation of the software system. This detailed design or implementation can be done either using an aspect-oriented language or a non-aspect-oriented language. In the former case, aspects and pointcuts identified at the architectural level help to write pointcuts and aspects in our aspect-oriented language. In the latter case, the designer or developer would need to manually weave the aspectual components with the components they crosscut and the benefits achieved by the aspect-oriented decomposition will lose, so this option is not recommended at least it is strictly required.
(2) The CAM aspect-oriented architecture can be used to automatically generate (see [44]a,2008b)) a DAOP-ADL description, which is used to run an application on the DAOP platform [85,43]. The DAOP platform is a distributed component and aspect based platform, where aspects and components are dynamically composed at runtime using the information provided by the DAOP-ADL aspect-oriented architectural description language [84].
(3) The CAM aspect-oriented architecture can be used to automatically generate, by means of model transformations, a UML design model for the application [103]. In this case, the model transformation is on charge of weaving the aspects with the components these aspects crosscut, producing a non-aspect-oriented UML model as output.
(4) The heuristics and rules described in [90] or [24] can be used to manually derive a Theme/UML [28] design model from a CAM aspect-oriented architecture. In this case, the separation of concerns achieved at the modelling level is preserved at design time, as we are using an aspect-oriented design language. More efficiently, this Theme/UML design model can also be automatically obtained from the aspect-oriented architecture using model transformations, such as described in [103].
(5) The CAM aspect-oriented architecture can also be used to automatically generate code for different languages [48], both aspect-oriented (AspectJ and JBoss AOP), as non-aspect-oriented ones (Java).

Next section illustrates the different steps in our approach by applying it to an Automatic Toll Collection System case study.

5. Case study: the Portuguese automatic toll collection system

This section illustrates our approach, depicted in Fig. 3, by using an example inspired in the Portuguese Automatic Toll Collection System[9] [87]. A summary of the basic functionalities of the system is described as follows.

In a road traffic pricing system, drivers of authorised vehicles are automatically charged at toll gates. The gates are placed in special lanes, called green lanes. A driver has to install a device (a gizmo) in his/her vehicle. The registration of authorised vehicles includes the owner's personal data, bank account number and vehicle details. The gizmo is sent to the client to be activated using an ATM that communicates with the system when the gizmo is activated. The toll gate sensors read data from the gizmo. The information read is stored by the system, and used to debit the respective account. When an authorised vehicle passes through a green lane, a green light is turned on, and the amount being debited is displayed. If an unauthorised vehicle passes through it, a yellow light is turned on and a camera takes a photo of the number (later used to find the owner of the vehicle). Certain toll gates, known as single tolls, charge according to the type of vehicle and others, the two point toll gates, charge an amount according to the distance travelled. In this case, the entry and exit points are registered and the vehicle owner debited.

We have selected this case study because it is simple to understand, contains a lot of interesting functional and non-functional requirements, providing appealing model elements that can be used to illustrate all the potentialities of our approach.

[9] ViaVerde website, http://www.viaverde.pt/ViaVerde/vPT/A_via_Verde/O_Que_E/.

5.1. Requirements identification

For this paper, we are assuming that an aspectual requirements specification does not yet exist. Therefore, the first step is to gather functional and non-functional system requirements, which will be used to drive the architecture design. As already stated, this can be achieved by using any of the currently existing AORE approaches. The only restriction imposed, as commented before, is related to the sea-level granularity of the functional requirements description [29], to guarantee that each requirement contains a single interaction with the system, plus a set of associated aspectual requirements. For this paper, we have chosen the Aspect-Oriented Requirements Analysis (AORA) [20,98,18] approach, since we have previous experience in handling it. However, as we said, other approaches could be selected.

As summarized in Section 2.2, AORA, like most of the AORE approaches, starts by identifying concerns, then specifies them and later composes them. This approach starts by identifying the kite-level [29] concerns, which are:

- `PassSingleToll`: this handles usage details in a single point tollgate. If the vehicle is registered, a light turns green, the amount to be debited in the owner's account is displayed and the passage is stored. Otherwise, the light turns yellow and the vehicle registration number is photographed.
- `EnterMotorway`: this handles vehicles joining a motorway. If the vehicle is registered, a light turns green and the entry data is stored. Otherwise, the light turns yellow and the vehicle registration number is photographed.
- `ExitMotorway`: this handles vehicles leaving the motorway. If the vehicle is registered and entered the motorway using the system, a light turns green and the amount to debit is displayed. Otherwise, the light turns yellow and the vehicle registration number photographed.
- `PayMonthlyBill`: this bills the system users on a monthly basis. Payments are done using an automatic bank direct debit from the vehicle owner's account.
- `ResponseTime`: the system needs to react in time in order to read the gizmo identifier, to turn on the light (to green or yellow), to display the amount to be paid, etc.
- `Authenticity`: the system is offered to registered users only. This means that vehicles need to be recognised and the system will react accordingly.
- `Integrity`: the system needs to ensure the message is not corrupted between leaving a sender component and arriving at the receiver.
- `MultiAccess`: the system needs to be able to handle multiple users simultaneously.

Here, our goal is not to apply AORA fully, but simply use it as a guide to obtain the final specification needed. (Note that many details in AORA are needed to indentify and manage conflicting situations, which we are ignored here. This is one of the major reasons why we will not build here a complete AORA specification.)

So, the next step is to refine these concerns to sea-level granularity [29]. For example, the `EnterMotorway` concern is further decomposed into the following sea-level concerns: (1) `AuthoriseVehicle`, (2) `RecordVehicleEntry` and (3) `ShowGreenLight`. This means that the behaviour specification of `EnterMotorway` is obtained from composing the three smaller concerns. The composition will establish the order in which its constituent elements are satisfied. In AORA, this composition is specified using a set of operators that allows, for example, the definition of: (1) sequencing of actions (through the enable ('≫') operator), (2) alternative actions (through the choice ('[]') operator); and (3) parallel actions (through the parallel ('||') operator) [19,20,98,18].

In our example, the composition rule for `EnterMotorway` will define that only if `AuthoriseVehicle` succeeds, the vehicle is authorized to use the Green Lane system. The `RecordVehicle Entry` and `ShowGreenLight` must then be satisfied, which might happen in parallel (although parallel execution is not mandatory). `EnterMotorway` is affected globally by the aspectual concern `ResponseTime`. This means that the whole `EnterMotorway` behaviour has to be performed in a fixed amount of time. This constraint affects the smaller component scenarios. Thus, using AORA operators, the `EnterMotorway` scenario will be defined as a composition of its constituent elements as follows:

```
EnterMotorway: AuthoriseVehicle ≫
(RecordVehicleEntry || ShowGreenLight)
```

The `AuthoriseVehicle` scenario is responsible for identifying a vehicle by reading its gizmo each time it passes through a toll gate. Additionally, it has to satisfy a set of quality attributes, or non-functional concerns. In this paper, a relevant subset of all non-functional concerns associated with this scenario is considered. In particular, the `AuthoriseVehicle` scenario needs to guarantee that only authenticated drivers use the system and it needs to be executed within a short period of time (`response-time`). Therefore, this scenario will be defined by the functional concern `Identification` and the non-functional concerns `Authenticity` and `ResponseTime`. The AORA process identifies `Authenticity` and `ResponseTime` as crosscutting concerns at the requirements level.

The `RecordVehicleEntry` concern is responsible for sending authorized vehicle entry data (vehicle identification, entry time and toll gate identification) to the Green Lane central system. Additionally, this concern must guarantee the `Integrity` of the data sent to the central system within a short time frame, so that it contributes to the global `ResponseTime` restriction of `EnterMotorway`.

The `ShowGreenLight` concern is responsible for showing a green light to the driver of an authorized vehicle. The light must be on for a long enough period so that it can be seen by the driver before s/he leaves the toll gate area.

These requirements are expressed in a machine-readable model in the next subsection.

5.2. Aspect-oriented requirements modelling

Each of the concerns identified need to be further described using scenarios. Once this has been done, the next step is to model them in UML, to obtain a machine-readable model that an automatic model transformation can process [68,77]. We have developed the Aspectual Scenario Modelling (ASM) UML Profile, similar to [28], to address this goal. Another related approach can be found in [7].

The ASM Profile allows the construction of requirements models based on scenarios where sky-, kite- and sea-level scenarios are modelled as packages. For the case of sea-level scenarios, each package contains non-crosscutting functional concerns that describe a single interaction with the system by means of a sequence diagram (communication diagrams can also be used, as they are equivalent to sequence diagrams). These non-crosscutting functional concerns are affected by zero or more aspectual requirements.

An aspectual requirement is modelled as a template classifier, following an approach similar to Theme/UML [28]. This classifier describes the aspectual requirement in a general and abstract way, referring only to template parameters instead of specific elements of the non-crosscutting functional concerns. For instance, in Fig. 4, `ResponseTime` is described by means of generic messages called `messageA` and `messageB`, instead of specific messages taken

from the non-crosscutting functional concern. Authenticity is described textually, using messageA and messageB as template parameters of this textual description. How aspectual requirements are modelled does not affect their transformations. Only the name of the aspectual requirement and its parameters are required for the model transformations, as we will explain in the next section.

Aspectual requirements are composed with non-crosscutting functional requirements by means of «bind» relationships, also inspired by Theme/UML [28]. These relationships instantiate the aspectual requirements for particular non-crosscutting functional concerns, specifying how and where the aspectual requirements apply, providing actual elements of the functional requirements. This technique improves the reusability of aspectual requirements, as they can be defined outside the packages that contain the non-crosscutting functional requirements they affect. Thus, this enables reuse of aspectual requirements that can be imported by any package. Considering that most of these requirements (e.g. Integrity, Authenticity, Persistence and Encryption) reappear across different applications, they are to be placed in a model library for reuse, creating a catalogue of reusable aspectual requirements.

It is possible to express composition relationships between non-crosscutting functional requirements and aspectual or crosscutting, requirements using quantification operators in the ASM Profile. This avoids having to create a bind relationship for each non-crosscutting functional requirement that is crosscut by an aspectual requirement. Thus, it is possible to express patterns that specify, for example, "crosscut any kind of message, with any kind and number of arguments, sent to the Toll Gate". Nevertheless, the techniques used for this purpose, in order to be compliant with the UML standard and most of the UML tools, require high expertise of the UML language. Thus, and to avoid overwhelming the reader with unnecessary details, we have not used them in this article. We refer the interested reader to a technical report that describes this technique in-depth [42].

Finally, kite- and sky-level scenarios are modelled using composition of sequence diagrams, using the UML 2.0 fragments, such as referencing (ref) alternative (alt), parallel (par) and optional (opt) (see Fig. 7). The resulting sequence diagram can also contain constraints, which ensures that the result of the composition of the sea-level scenarios satisfies some aspectual requirements.

Going back to our example, Fig. 4 shows how the AuthoriseVehicle scenario is modelled according to the ASM Profile. Fig. 4(left) models the simple functional concern Identification by means of a sequence diagram. The ResponseTime between two messages is described in abstract using a UML sequence diagram plus a time constraint t, as shown in the central part of Fig. 4. Authenticity is described textually (see Fig. 4, right) as follows: "To guarantee Authenticity between messageA, sent by A to B, and messageB, sent by B to A, some extra mechanisms have to be provided", where messageA and messageB are template parameters. We would like to remind the reader once more that these aspectual requirements are modelled in separate packages and then imported by the package modelling scenarios, avoiding scattered and tangled representations of aspectual requirements.

Finally, the «bind» relationships bind(who, i_am, tl) and bind(who, i_am) instantiate the aspectual requirements ResponseTime and Authentication templates. This instantiation provides actual values (e.g. who, i_am, tl), coming from the functional concern Identification, to the formal parameters of ResponseTime and Authenticity (e.g. messageA, messageB, t).

Figs. 5 and 6 show the RecordVehicleEntry and ShowGreenLight scenarios modelled according to the ASM Profile.

As mentioned before, to obtain the full behaviour of the kite-level EnterMotorway, we need to define a composition rule that specifies how and where the sea-level scenarios must be

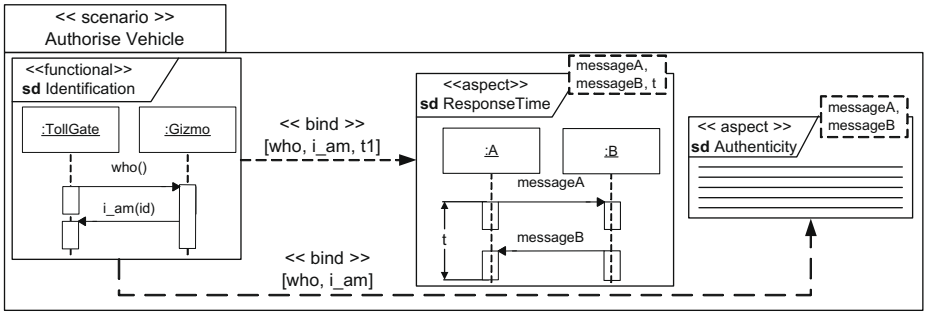

Fig. 4. UML representation of the Authorised Vehicle scenario.

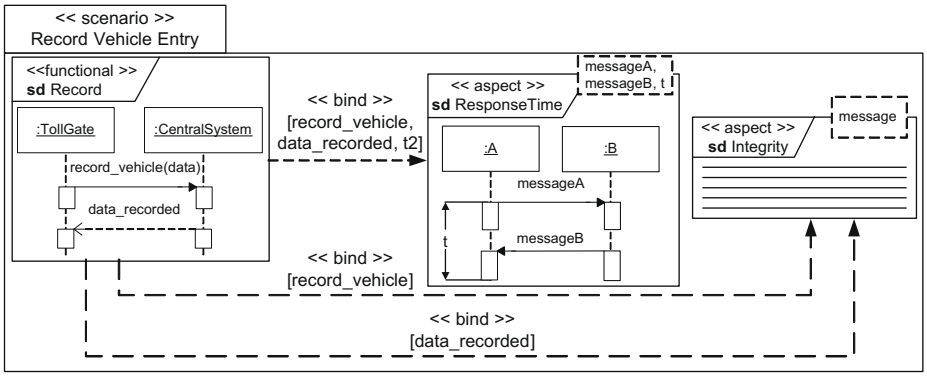

Fig. 5. UML representation of the RecordVehicleEntry scenario.

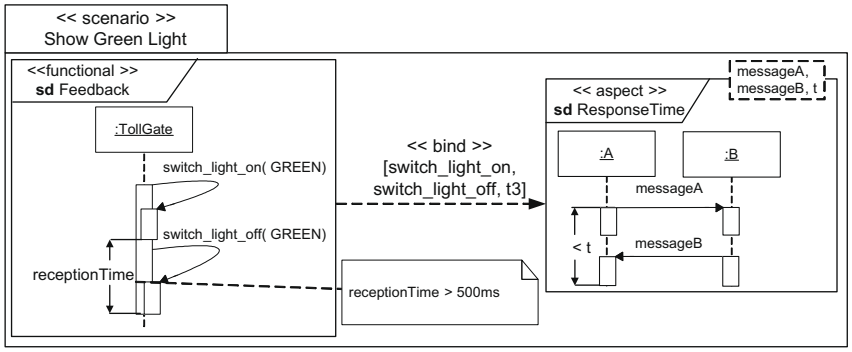

Fig. 6. UML representation of the `ShowGreenLight` scenario.

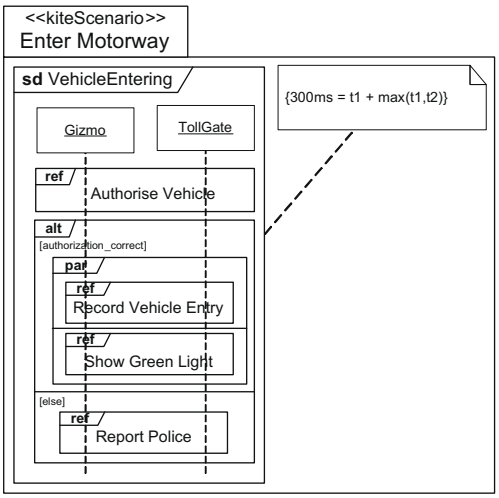

Fig. 7. UML representation of the `EnterMotorway` kite-level scenario.

composed together. This composition rule is illustrated in Fig. 7. This figure shows the `EnterMotorway` kite-level scenario as a result of the composition of its constituent sea-level scenarios. This scenario can be easily obtained from the AORA composition rule presented in the previous section, by applying the mappings proposed in [20,18]. The rule specifies a correspondence between AORA operators and UML operators for interactions, such as parallel or alternative. In our case, a global constraint indicates that the resulting `ResponseTime` has to be lower than a certain value (300 ms). This constraint is specified based on the `ResponseTime` values of the sea-level scenarios, `t1`, `t2` and `t3`, which implies that this aspectual requirement must be observed by each of them.

These aspect-oriented requirements models will be automatically transformed into an aspect-oriented architecture model, using pre-defined and reusable automatic model transformations. The next subsection explains how these pre-defined and reusable automatic model transformations work.

5.3. Creation of the model transformations

This subsection explains how model transformations have been designed to automatically generate an architecture from an aspectual requirements model. Our proposal is to use QVT (Query, View, Transformations) [86] to specify the automatic model transformations that allow the generation of aspect-oriented architectural models expressed in CAM, from the aspect-oriented requirements model expressed in ASM. This transformation is achieved in a bottom-up style, that is, from sea-level to kite-level scenarios. The transformation of the sea-level scenarios is defined through the rules listed in Table 1.

For instance, the message `who( )` sent from `TollGate` to `Gizmo` (Fig. 4, `Identification` functional requirement) would not be transformed step 2 (Table 1) since it is affected by an `Authenticity` aspectual requirement (among others). Then, in step 3 (Table 1), it is transformed into an augmented interaction with new elements added by the pattern (aspectual components in this case), which provides the required authentication support.

As mentioned before, several patterns that satisfy a given aspectual requirement might exist. For example, `Integrity` of messages sent between two entities can be supported by techniques such as Checksum (CRC), one-way hash algorithm (MD5, SHA1) and a digital signature [53,104]. It is the software architect's responsibility to select the appropriate transformations or solutions that best satisfy the aspectual requirements, taking into consideration trade-offs between these requirements and stakeholders needs.

Each stereotype «aspect»of the ASM Profile has a tagged value called `architecturalSolution`. Once the software architect selects a specific solution for an aspectual requirement, s/he sets the name of this tagged value to the name of the selected model transformation. This ensures that when generating the software architecture, only the transformation corresponding to the pattern of the selected solution is applied.

The transformation of kite-level scenarios does not require any special effort, as their structure is simply copied into the architectural model. Constraints attached to kite-level scenarios about the composition of smaller scenarios are also copied as composite scenarios into the architectural model. Thus, as the composite scenarios are simply copied, they will be correct if the result of the transformation of their constituent parts is correct, that is, if the result of transforming the sea-level scenarios that constitutes the kite- and sky-level scenarios is correct.

As commented before, the transformation rules have been specified in QVT and have been implemented in ATL as it will be explained in Section 6. Fig. 8 shows one of the QVT relations that specifies the rule for transforming functional requirements (step 1 in Table 1). Specifically, the QVT relation refers to the transformation of messages. In this case, for each message found in a sequence diagram modelling a functional requirements scenario, an interface, a provided and a required relationship and an operation are created in the architectural model. The pattern in the left-hand side of the figure specifies, in terms of the UML metamodel, that each time a message between two lifelines is found in the source model (left pattern), an interface with a specific name should exist. This interface is provided by the port associated with the component resulting from transforming the lifeline which receives the message (right pattern). This interface is required by the port associated with the component resulting from transforming the lifeline which sends the message (right pattern). The `when` clause, in a QVT relation, indicates that before satisfying this relation, the

Table 1
Transformation rules of the sea-level scenarios.

1. Functional concerns (described by means of sequence diagrams) are transformed into architectural artefacts, according to the following steps:
   a. For each lifeline representing a different concept in the sequence diagram, a component with a port is added to the architecture model. The name of the component is the name of the type of the lifeline.
   b. If a lifeline A receives a message in the requirements model, an interface IA is incorporated into the architectural model and a provided relationship is established between the component A (resulting from transforming the lifeline A following the rule 1.a) and the newly created interface IA. The operation invoked by the message is added to the interface IA, with their corresponding parameters.
   c. For each lifeline B that sends a message to a lifeline A, a required relationship from the corresponding component B (created previously by transformation of lifeline B fulfilling rule 1.a) and the newly created interface IA is added to the architectural model (according to rule 1.b).

   These steps create the structural view of the architectural model, describing the components that comprise the architecture and their connections.
2. Each intertion (i.e., the sequence diagram) describing a functional concern is transformed into an incomplete interaction in the architectural model (i.e., an incomplete architectural sequence diagram). This interaction contains the transformation of all the messages that do not appear as parameters in any of the bind relationships, i.e., that are not affected by any aspect. This step creates the skeleton of the architectural view of the description. Messages affected by aspects are not included as they might be intercepted, reified or modified by the aspects.
3. Aspectual requirements and the messages affected by them are transformed into architectural artefacts using pattern-based transformations, where a certain pattern encapsulates the design of a suitable solution for the aspectual requirement. This pattern is instantiated using the parameters of the bind relationship that composes the aspectual requirement with the functional requirement. This transformation step only requires the name of the aspectual requirement, which determines the pattern to be injected and the actual values of the bind relationship, which provides the values for instantiating the pattern.

Note that as mentioned in the previous section, the way the aspectual requirement is modelled does not affect the transformation process. Thus, the description of an aspectual requirement serves for documentation only. This last step completes the behavioural view created in the second step.

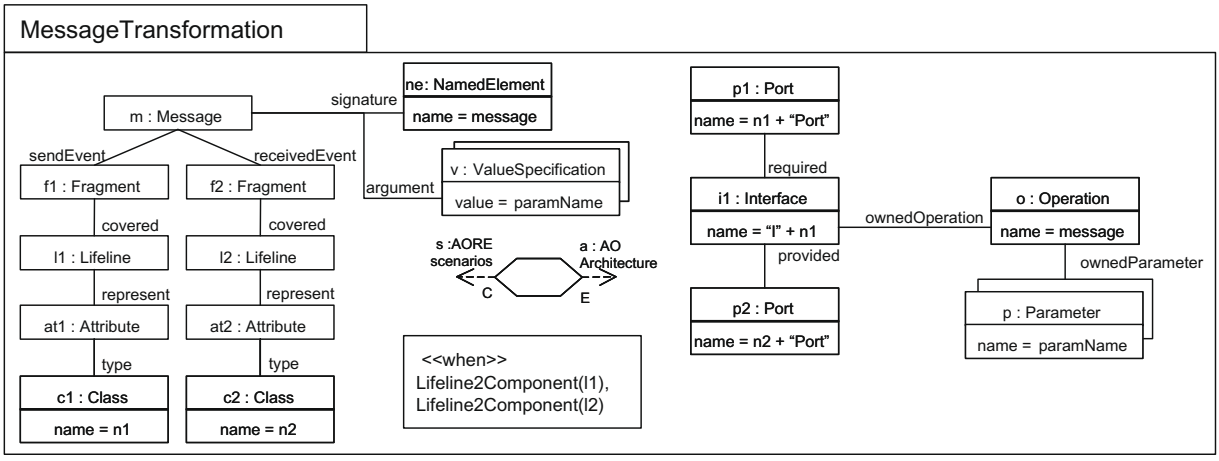

Fig. 8. Message transformation specified in QVT.

`Lifeline2Component` relation has to be satisfied by lifelines `l1` and `l2`. It ensures that components and associated ports have already been created when the `MessageTransformation` relation is trying to be satisfied.

Fig. 9 depicts one pattern-based transformation for the aspectual requirement `ResponseTime`. At the architecture level, the pattern only preserves this aspectual requirement, which is passed to later phases of the software development lifecycle. It adds a temporal constraint to the appropriate behavioural view of the architecture to preserve the aspectual requirement. By preserving the aspectual requirement, designers, programmers and hardware buyers are forced to observe this constraint when implementing the system architecture and, therefore, the aspectual requirement is not lost at the architectural level. The main elements for performing this transformation are shown in grey. Basically, each time, within a scenario, a `bind` relationship is found associating a functional requirement with a `ResponseTime` aspectual requirement (top pattern), and this specific solution for `ResponseTime` has been accepted, a UML temporal constraint for the messages influenced by the `ResponseTime` aspect is injected into the architectural model (bottom pattern). For the injection of the temporal constraint, actual values of the `bind` relationship parameters `mess1`, `mess2` and `timeValue` are used.

It should be noted that these model transformations do not need to be developed each time our approach is applied. They are developed once, and then, they become reusable for different projects. Transformation for non-aspectual requirements are reusable for any kind of project, whereas transformations for aspectual requirements are reusable whenever the same aspectual requirement appears in another project and the software architect wants to apply the same pattern solution. So, if a software architect can find satisfactory model transformations for all the aspectual requirements in his project, he does not need to develop any new model transformation, and if s/he trusts in our work, s/he does not need to understand the process described in this section, as these model transformations are automatically executed.

5.4. Selection of model transformations

This section describes the solutions selected for transforming the aspectual requirements identified and modelled in the previous subsections. We analyse, for each aspectual requirement, alternatives available, decisions adopted and the rationale behind these decisions.

5.4.1. Authenticity

Authenticity requires the verification of the identity of entities involved in a communication. There are several algorithms to achieve this requirement [104]. However, a decision on the specific algorithms chosen should not be made yet, since:

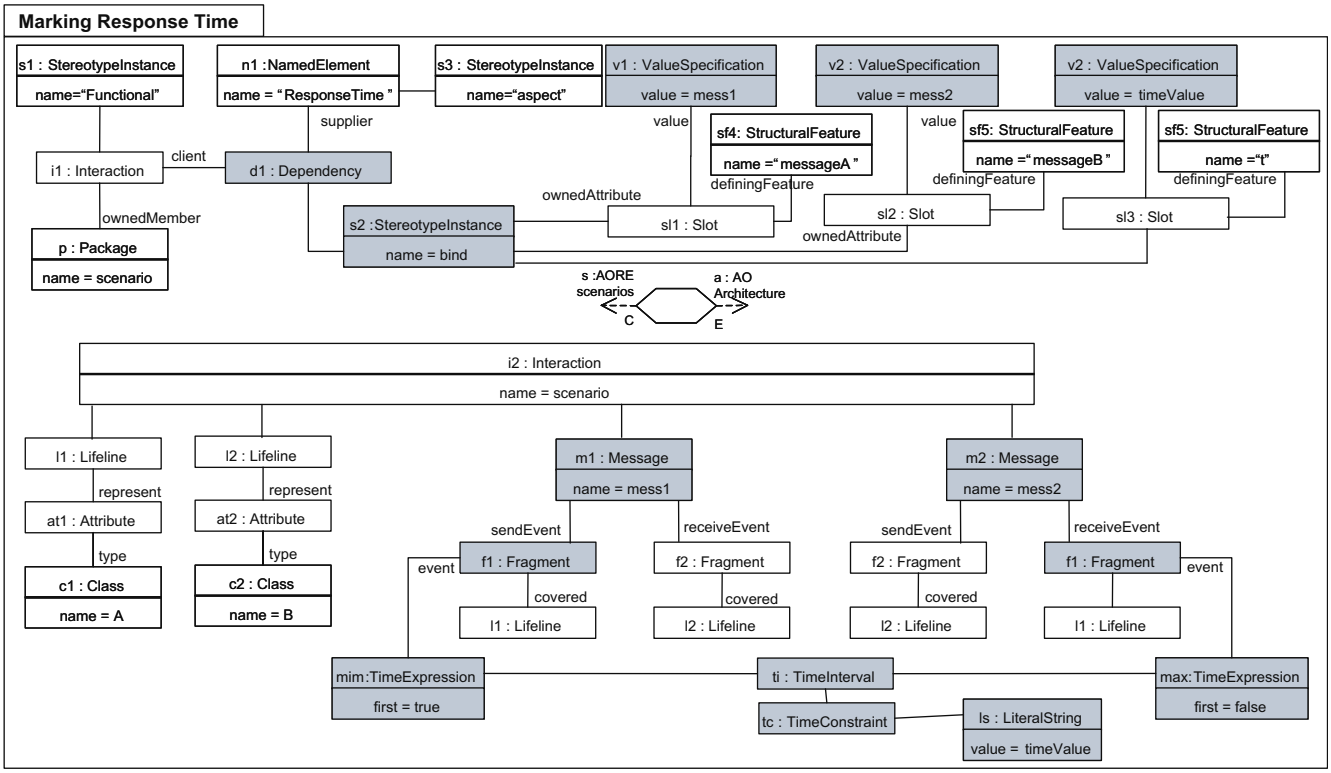

Fig. 9. Response Time Pattern Transformation specified in QVT.

(1) It may involve low-level details that are irrelevant at the architectural level.
(2) It might not be possible to select an authentication algorithm until all the trade-offs involving Authenticity are solved (and it may be not possible to solve some of them at the architectural level, such as, for instance, the acquisition of specific embedded microprocessors).
(3) It may require unavailable expertise in the field at this stage.

Aspect-oriented techniques allow us to stay at a level of abstraction where we can ignore how base components must deal with Authenticity. Since aspects are not invoked by components, they are transparently executed during base component interactions. Thus, it is possible to create a generic pattern for injecting Authenticity in the architecture.

Algorithms for ensuring authenticity are based on introducing two new (aspectual) components for dealing with Authenticity. We call these aspectual components `AuthenticationSend` and `AuthenticationReceive`. When one of the base entities sends a request to the other base entity, this request is intercepted by the `AuthenticationSend` aspectual component. Then, a dialogue is initiated between the Authentication components, and the request is forwarded to the corresponding base entity. When the answer to the request is sent, this is intercepted again, a new dialogue is initiated between the Authentication entities and if the authentication process ends successfully, the answer is forwarded to the requester entity. Otherwise, the system is informed on the error and the exception is managed adequately.

The important issue here is that the interactions related to Authencity are encapsulated in a separate module, outside the components. This can be done since it does not affect the base components communications. This makes the component oblivious of the Authenticity aspect, or any other aspect being applied to it. (Next subsection will show an example of an application of this pattern, in Fig. 11.) Thus, the transformation encapsulating this pattern is selected to be applied to the architectural model. If this transformation would not have been created before, we would need to create it for this project and it would be available in a transformation repository for other projects.

5.4.2. Integrity

`Integrity` requires mechanisms to ensure that the message is not corrupted between the moment it leaves the sender and the moment it reaches the receiver. Traditional algorithms [53,104], such as MD5, RSA, hash functions and CRC, for example, add extra data to the message before sending it, and use mathematical mechanisms to allow the receiver to check if the message was corrupted. The sender is notified about the status of the message, and if corruption is detected, the sender resends the message.

Injecting Integrity into a software architecture creates similar issues to those introduced with Authenticity. Thus, the solution adopted is analogous. We introduce two `IntegrityHandler` aspectual components. These components intercept the messages whose Integrity must be ensured and they dialogue according to a certain integrity algorithm. The decision about what integrity algorithm should be selected is postponed to later phases, where, for instance, all timing constraints are better known. Thus, the transformation encapsulating this pattern is selected to be applied to the architectural model.

5.4.3. Response time

There are several good solutions available to handle ResponseTime. For example, to improve performance of a system attending requests, a caching strategy can be adopted. In such a case, aspects can be used to inject caching support. To achieve this goal, some implementation languages and specific hardware can help more than others. This is not, however, an architectural decision and should, therefore, be postponed.

In our example, what we can do at the architectural level is described as follows. The Gizmo identification needs to be achieved

within a certain period of time. (A gizmo is a small electronic device placed in the vehicle windscreen.) We can attach time constraints to the design specification to guarantee that the component is implemented in such a way that that restriction is guaranteed. Thus, the transformation encapsulating this pattern is selected to be applied to the architectural model. Fig. 9 depicts one of the QVT relations used to specify this model transformation.

The following section shows the results of applying the automatic model transformations, that encapsulate the solutions described above, to the requirements models.

During this step, the software architect must identify the most appropriate solution from a set of the potential solutions for each aspectual requirement for the system under development. This task is not specific for our approach, and it should be carried out in any software architecture designing process. But, using our approach, the software architect needs only to understand the rationale behind each solution and its trade-offs, e.g., a higher security at the cost of less performance. But, since the pattern solution is automatically applied by means of model transformations, the software architect does not need to understand the details of each pattern solution, which can be simply abstracted.

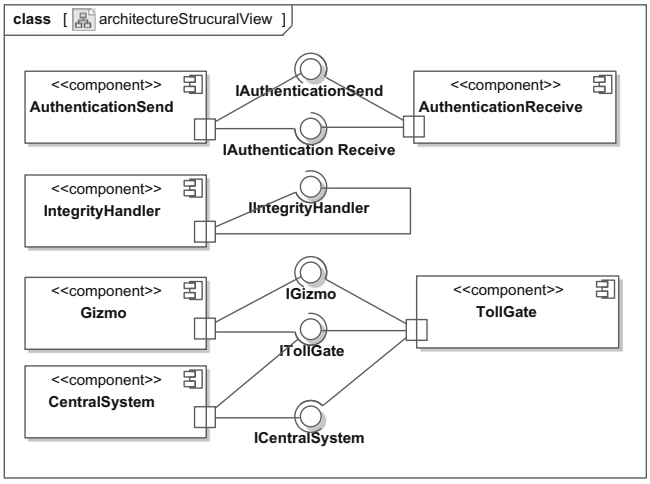

Fig. 10. Structural view of the architecture obtained by model transformation.

5.5. Obtained aspect-oriented architectural description

The architectural model, which is automatically obtained from applying the transformations described in the previous section to the requirements specification, is expressed in the UML 2.0 Profile for CAM [85]. Fig. 10 depicts the structural view of the generated architecture in terms of components, ports, interfaces and connectors. Figs. 11–13 represent the architectural behavioural views for `AuthoriseVehicle`, `RecordVehicleEntry` and `ShowGreenLight`. Since `EnterMotorway` is similar to the one in Fig. 7, it is not shown here.

When the automatic transformation process is applied to the scenarios in Figs. 4–6, `TollGate`, `Gizmo` and `CentralSystem` lifelines of the functional requirements `Identification`, `Record` and `Feedback` are transformed into the components `TollGate`, `Gizmo` and `CentralSystem`. The messages `who( )`, `i_am(x)`, `record_vehicle(data)`, `data_recorded( )`,`switch_light_on(light)` and `switch_light_off(light)` are transformed into operations of the interfaces `IGizmo(who)`, `ITollGate(i_am, data_recorded, switch_light_on, switch_light_off)` and `ICentralSystem(record_vehicle)`, respectively. These interfaces are associated with the previously created components, as shown in Fig. 10. The aspectual components `AuthenticationSend`, `AuthenticationReceive` and `IntegrityHandler` are also introduced in the architecture to satisfy the aspectual requirements `Authenticity` and `Integrity` by means of pattern-based transformations. The following paragraphs describe how each interaction is obtained.

5.5.1. Authorise vehicle scenario

Fig. 11 shows (a) the interaction between components that fulfils the functional requirement `Identification` (messages `who()` and `I_am(x)` outgoing and incoming from/to `GizMo` and

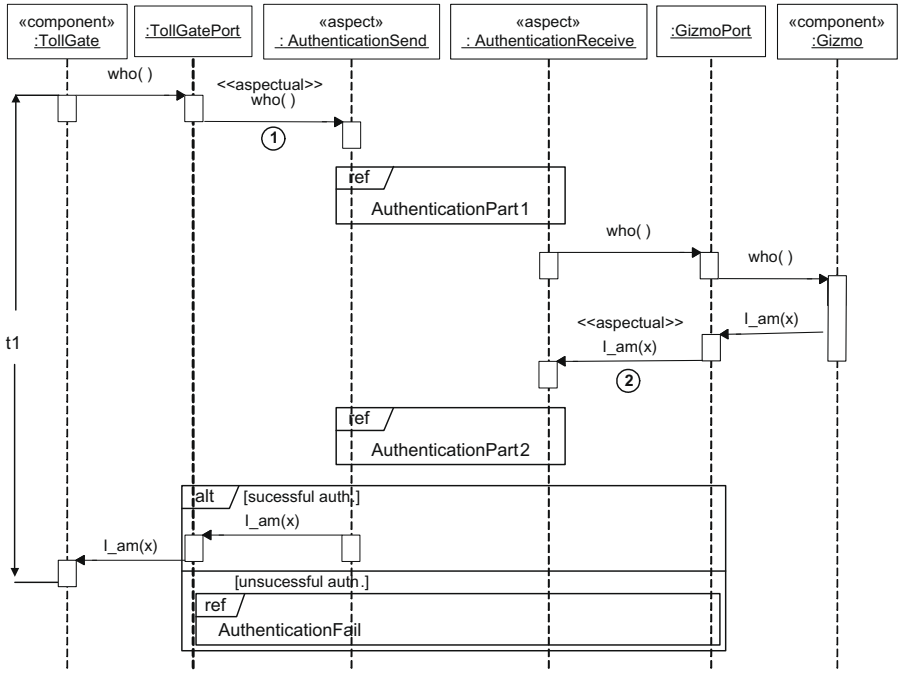

Fig. 11. Architectural behavioural representation of the `AuthoriseVehicle` scenario.

TollGate components) and (b) the aspectual concerns Authenticity and Response Time. This figure illustrates how the pattern for accomplishing authenticity has been introduced. When the TollGate component sends a who( ) message through its port, this message is intercepted by an AuthenticationSend aspect (represented by the stereotyped message «aspectual» (label 1 in Fig. 11) sent from the component to the aspect). The AuthenticationSend aspect initiates a dialogue with the AuthenticationReceive aspect, for checking Authenticity. Then the AuthenticationReceive aspect delivers the request to the Gizmo component through its port. Next, the Gizmo component sends the answer to the request through its port. This answer is intercepted by the AuthenticationReceive aspect (label 2 in Fig. 11). This initiates a new dialogue with the AuthenticationSend aspect. The purpose of these two dialogues between aspects is to check the authenticity of the answer, through a specific algorithm. If the authenticity checking is successful, the AuthenticationSend aspect delivers the answer to the TollGate component. If the authentication fails, an error dialogue is executed.

As a result of executing the model transformation for ReponseTime, partially depicted in Fig. 9, the time constraint t1 is introduced between the sending of the who( ) message and the reception of the i_am(x) message. Please note that the temporal constraint also includes the dialogue between aspects for ensuring Authenticity. Therefore, the requirements trade-offs between ReponseTime and Authenticity are preserved at the architecture level.

5.5.2. Record vehicle entry

Fig. 12 illustrates the architectural behavioural description obtained after processing the scenario RecordVehicleEntry. The interaction between components satisfies the requirements of the RecordVehicleEntry scenario. This figure shows the pattern that provides this generic solution for Integrity, instantiated for the messages record_vehicle(data) and data_recorded. As a result of injecting this pattern in the software architecture, an IntegrityHandler aspect is added to both sides of the communication. This aspect encapsulates the coding and decoding functionality, so that it can be used both for sending and receiving. This aspect intercepts the message sent from the ToolGate to the CentralSystem (Fig. 12, label 1) and its return (Fig. 12, label 2). The IntegrityHandler at the toll gate side adds extra data to the message to ensure integrity and sends the resulting message to the IntegrityHandler aspect on the other side of the communication. If this latter aspect detects that the message is corrupted, it requests the resending of the message. As it can be noted, the decision about which specific integrity algorithm should be used is left to the domain designers experts.

As a result of executing the model transformation for ReponseTime, the time constraint t2 is introduced between the sending of the record_vehicle( ) message and the reception of the data_recorded message.

Again, please note that the temporal constraint also includes the dialogue between aspects for ensuring Integrity. Therefore, the requirements trade-off between ReponseTime and Integrity are also preserved at the architecture level.

5.5.3. Show green light

Fig. 13 depicts the architectural description of the ShowGreenLight scenario. The interaction between the involved components satisfies the functional Feedback scenario. The aspectual requirement ResponseTime, which influences this scenario, is also added. As previously, the transformation places a temporal constraint (t3) to satisfy ResponseTime.

5.5.4. Enter motorway

The EnterMotorway scenario is copied directly to the architecture with the only difference that now lifelines represent instances of components. A figure showing the results of this transformation has not been included, for brevity.

5.6. Summary

The result obtained from applying our approach to the Toll System case study is an aspect-oriented architecture design. This architecture, as will be discussed in Section 7 and as reported in [30,35,36,45,49,71,78,92,105], exhibits a better modularization which contributes to a better system maintenance and evolution. This aspect-oriented software architecture is obtained (semi) automatically. The guidance of the software architect is only required for selecting those solutions for the aspectual requirements that better satisfy the user needs.

Ideally, a repository for model transformations should be provided. If we assume that the required model transformations are available in a model transformation repository, the aspect-oriented software architecture is obtained only with the effort associated to both constructing the scenario models and deciding which solutions must be used to satisfy each aspectual requirement. Thus, the manual work required for creating the software architecture is saved.

Finally, and as commented before, trade-offs between aspectual requirements are preserved at the architectural level.

6. Tool support

Whenever possible, we have used standard tools and languages. Since standards are being adopted in the development of industrial applications, we expected, since the beginning, to find good tools and to decrease the classical learning curve, avoiding the need to learn new languages and notations.[10] Furthermore, as our approach is independent of the AORE approach used, the requirements engineer can select the process that s/he prefers, thereby decreasing the adoption effort required. Of course, the tool support for this first stage depends on the tools provided by the chosen AORE approach.

6.1. Tool support for constructing the aspect-oriented models

The aspect-oriented scenario model and the architectural model are based on UML 2.0 Profiles, ensuring that common UML tools can be used. The aspect-oriented scenario model serves as input to the automatic model transformation, which returns an aspect-oriented architectural design model. Transformation tools manage UML models using their XMI[11] representation, although intricacies of the XMI format are hidden from the user by model transformation languages. This means that any UML 2.0 tool with an XMI import/export facility can be used. The wide range of existing UML modelling tools[12] facilitates the developers' choice.

6.2. Tool support for specifying and implementing the automatic model transformations

Although QVT is an OMG standard, there is still a lack of stable tools supporting its languages. Nonetheless, we have specified transformations in QVT for two reasons:

[10] The importance of using standards for a successful adoption of MDD is disucussed in [93].

[11] XMI is a standard to serialize models according to a machine-readable XML format.

[12] http://www.uml.org/#Links-UML2Tools.

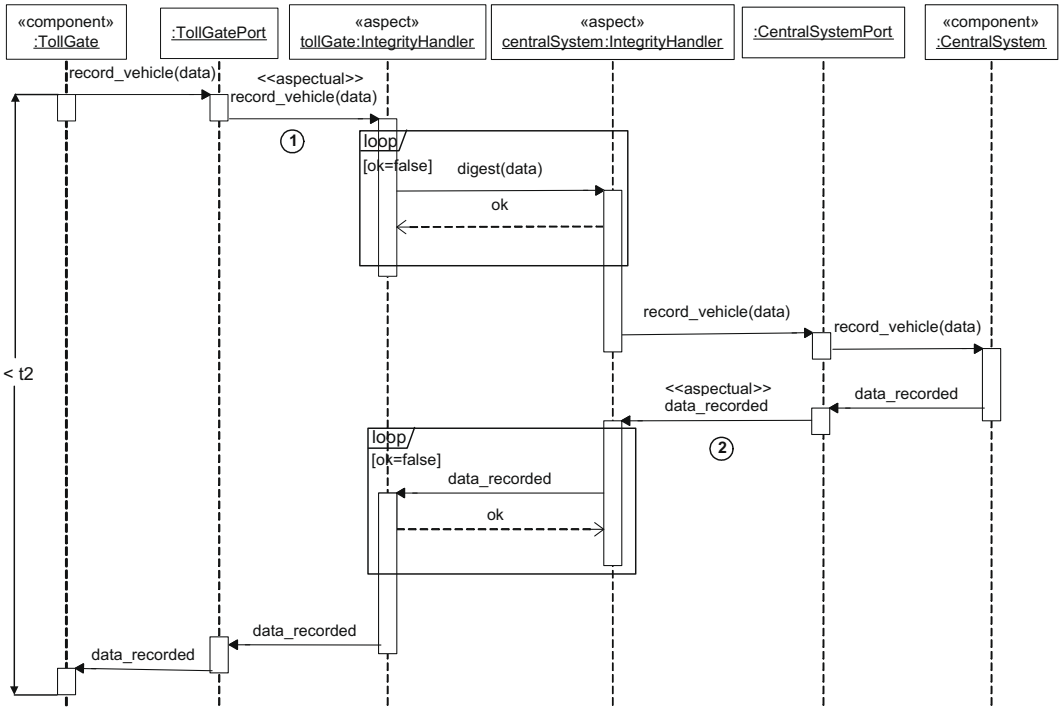

Fig. 12. Architectural behavioural representation of the `RecordVehicleEntry` scenario.

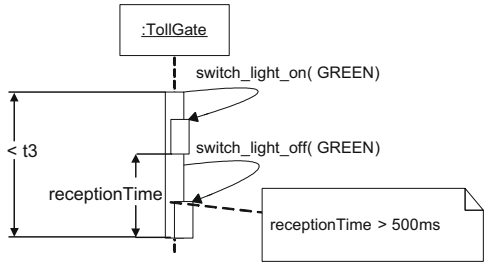

Fig. 13. Architectural behavioural representation of the `ShowGreenLight` scenario.

1. When stable tools are released, it will be possible to implement QVT-relations directly.
2. Meanwhile, QVT-relations serve as a good guide to specify how to implement transformations in other languages.
3. Although there are not stable tools 100% compliant with the QVT standard, there are stable languages similar to QVT, such as ATL [57,58], that can be used in the meanwhile.

Therefore, the QVT relations have been implemented in ATL.

7. Evaluation and discussion

This paper presents a (semi) automatic approach to generate aspect-oriented architectural designs from aspect-oriented requirements models. While the architectural models are expressed in the UML 2.0 Profile for CAM, the requirements models are expressed in the ASM UML 2.0 Profile. The transformation process ensures that the information contained in the requirements models is preserved at the architectural level. This helps to bridge the gap between aspect-oriented requirements and aspect-oriented architectures. This gap is even more noticeable in aspect-orientation than in traditional development techniques since, in addition to the classical misalignment between requirements specification and architecture design, aspect-oriented requirements engineering and aspect-oriented architecture design approaches have emerged in isolation from each other. Therefore, an automatic process to map requirements-level aspects into architectural-level aspects does not exist. This mapping is the major contribution of this paper.

This section evaluates the contributions of this paper and provides a critical discussion on benefits and pitfalls of our approach. The conclusions are a result of the experience gained after applying our proposal to three case studies: the TollGate system presented in this paper, an AuctionSystem[13] taken from [94] and an Online Book Store System[14] taken from [76]. Additional benefits gained from the joint use of AOSD and MDD technologies will also be discussed, even if they are not the main contribution of this paper. In particular:

1. The aspectual requirements identified during AORE will be reflected in the derived architecture.
2. The MDD transformation of aspectual requirements into architectural aspects is done automatically, alleviating software architects' task.
3. An improvement on modularization is obtained due to the use of aspect-orientation.

Section 7.1 evaluates each specific contribution contained in this paper and Section 7.2 analyses further the benefits from the joint use of AOSD and MDD technologies. Finally, Section 7.3 debates the applicability and scalability of our approach.

7.1. Evaluation of the transformation process

A summary of the evaluation results discussed in this section is shown in Table 2. The remaining of this subsection explains the contents of this table.

[13] http://www.lcc.uma.es/~pablo/CaseStudies/AuctionSystem/.
[14] http://www.lcc.uma.es/~pablo/CaseStudies/OnlineBookStore/.

7.1.1. Automatic generation of aspect-oriented architecture designs from aspect-oriented requirements specifications

The transformation rules described in Section 5.3 (QVT transformations) specify the main steps for generating aspect-oriented architecture designs, expressed in the UML 2.0 Profile for CAM, from aspect-oriented requirements models, expressed in the ASM UML 2.0 Profile. This transformation process has been specified in QVT, which ensures that it can be automated. The architectures generated by the automatic transformations are equal to the ones generated manually. This is a good indicator that the automatic transformations work well.

The transformation of aspectual requirements into architectural aspects, which is the main goal of this paper, has worked adequately for the three case studies, which contained eight crosscutting concerns in total (see Table 2). The transformations presented in this paper (Section 5.3, Figs. 9 and 10) work for `Authenticity`, `Integrity` and one of the possible solutions for `ResponseTime`. These transformations were reused in different case studies. In particular, `Authenticity` was (re)used in the Auction System, `Integrity` in the Online Book Store and `ResponseTime` in both. New crosscutting concerns, such as `Encryption` or `Persistence`, required the specification and implementation of new transformations, which were defined according to the technique presented in Section 5.3. However, once these new transformations have been specified and implemented, they can be incorporated into the repository of transformations for aspectual requirements and be reused across applications. These automatic transformations are reusable since they only depend on the name and the interface (parameters) of an aspectual requirement, and not on any other elements of a requirements model.

Regarding the transformation of the non-aspectual requirements, this is an additional contribution that was required for transforming a whole system. From our experience, only for the Auction System case study a minor part of the generated architecture needed to be refactored after the transformation. In this case, one of the scenarios lifeline was BankAccount. Instead of having one component for each customer bank account, we opted for creating a new component, called `VirtualBank` for managing individual customers' accounts. This new component, not identified as any lifeline in the requirements specification, emerged during the architecture design, to improve design reusability and performance. Interested readers can find more details about this issue in the authors' previous work [90].

The discovery of this emerging component relies on a certain expertise and creativity of the software architect that was not captured by the automatic model transformation. In addition, with the current transformations, it is not possible to define several provided or required interfaces for a component. Nevertheless, these shortcomings only affect the transformation of the non-aspectual part of the requirements and architecture models, which is not the main goal of this paper. Likewise, for aspectual concerns, the automatic transformations for the non-aspectual information are reusable in different applications, since they do not depend on any specific model element; they rely on pure metamodel-based manipulations.

7.1.2. Requirements information and trade-offs are preserved at the architectural level

The use of the three case studies, and different combinations of crosscutting concerns for them, was helpful to check that all the existing information in a requirements model is automatically transformed into some kind of architectural elements. Additionally, it has also been checked that the trade-offs identified at the requirements level are preserved at the architecture level. These phenomena can be observed in Fig. 11, where the time constraint includes the Authentication process. Therefore, the authentication algorithm to be selected at design time must satisfy the time constraint. Thus, the trade-off between `ResponseTime` and `Authentication` is preserved in the architecture model. The same happens in Fig. 12, for `ResponseTime` and `Integrity`.

7.1.3. Independence of the aspect-oriented requirements engineering approach selected

It is stated in this paper that our approach is independent of the Aspect-Oriented Requirements Engineering approach selected. This is validated as: (1) AORA [19,20,98,18] has been used for eliciting requirements for the TollGate system; (2) Theme/Doc requirements models [28] taken from authors' previous work [90] were reused for the Auction System case study; and (3) a use case model taken both from authors' previous work [41] and existing literature [76] was used for the Online Book Store system.

These requirements specification had only to be refactored to address the constraint regarding the decomposition of the requirements to sea-level. However, the selection and use of different AORE processes had no influence in the results. Thus, we can conclude that our approach is independent of the AORE process selected.

So far, the discussion focused on the evaluation of the main goals and contributions of this paper. Next, the extra benefits obtained from the joint use of AOSD and MDD are examined.

7.2. Benefits of AOSD and MDD approaches for this work

7.2.1. AOSD improves modularisation

This claim is discussed both for the requirements and the architecture phases. Regarding requirements, currently, the aspect-oriented community lacks in-depth and empirical studies demonstrating that these new techniques improve requirements modularisation. However, initial works, such as those described in [89], point to an improvement in modularization. In our experiments, however, it is unclear when aspect-oriented models described in the ASM UML 2.0 Profile improve requirements modularisation. There are improvements regarding aspectual requirements reusability, since they do not contain any element that refers to non-crosscutting functional requirements (an aspectual requirement uses template parameter elements when it needs to refer to the requirement it crosscuts). The dependencies between non-crosscutting and aspectual requirements are dependencies established by means of «bind» relationships (see Figs. 4–6). Therefore, it can be stated that these requirements are less coupled, which leads to a better modularization.

This would lead to improvements in maintenance and in evolution if these requirements descriptions changed. Nevertheless, these descriptions are quite stable and rarely change. Indeed, they are often shared across different applications, and they can even be catalogued (such as in [26]), stored in some repository, and reused across different applications. The changes required to link the aspectual requirements to other application-specific requirements are done at the composition level, where the relationships among requirements are established.

The stability of the aspectual requirements is what adds value to our approach, since automatic transformations specified and implemented for an aspectual requirement may also be catalogued and reused across different applications. Since aspectual requirements in the ASM UML 2.0 Profile are less coupled with base requirements, we could reuse some of them in several case studies (see Table 2).We still need to undertake a similar study for aspectual functional crosscutting requirements, but we believe that this will show similar findings.

The purpose of using aspect-orientation at the requirements level is not only to improve modularisation. Instead, it is mainly to obtain an early identification of aspect candidates and to detect,

through projections, their influence on each other, allowing the identification of conflicting situations where particular trade-offs will be required [87,79,21]. In our case, if an AORE process was not used, crosscutting concerns would result in scattered and tangled architectural configurations, which would not be desirable as will be shown later. This means that we would need to refactor the software architecture to identify crosscutting concerns and encapsulate them into components. From our experience, this cost is higher than the cost required for identifying crosscutting concerns at the requirements level.

Unlike with the aspect-oriented requirements case, there are some empirical studies for aspect-oriented architecture which assess how aspect-orientation improves modularisation in software architecture ([30,35,36,45,49,71,78,92,105]). Sant'Anna et al. coupling and cohesion metrics [92] have been used to compare the generated aspect-oriented architectural models with their non-aspectual version. These metrics measure several architectural qualities related to modularity, such as component cohesion and coupling. Table 3 explains how these metrics work. The architecture designs for two of the case studies (the Auction System and the Online Book Store) can be found in our previous work [44,41] and the architecture for the Toll Gate system is the contained in this paper.

The values in Table 4 represent the average results for the components in the architecture and the crosscutting concerns in Table 2. The lower these numbers are the better; that is, a high number represents a low coupling or a high cohesion. For some of these metrics (e.g., E.C. *Efferent Coupling*), the aspect-oriented version shows improvements as compared to its non-aspect-oriented counterpart. In other cases (e.g., CDAI), both architectures have similar results.

The improvement in some metrics is due to an increase of the cohesion and a decrease of the coupling between architectural components. This is explained for the Toll Gate system using Fig. 14. Here, the `Gizmo` component is explicitly connected to the component `AuthenticationSend`, the `TollGate` component is connected to `AuthenticationReceive` and `IntegrityHandler`, and `CentralSystem` is connected to `IntegrityHandler`.

Table 2
Evaluation results summary.

Evaluation items	Case studies		
	Toll Gate System	Auction System	Book Store System
Aspectual concerns (AC) addressed	Authenticity, Integrity, Response Time	Monitoring, Transaction Management	Encryption, Persistence, Currency Conversion
AC transformations reused	Encryption	Authenticity, Response Time	Integrity, Response Time
New AC transformations defined as in section 5.3	Authenticity, Integrity, Response Time	Monitoring, Transaction Management	Encryption, Persistence, Currency Conversion
Transformation of Non-aspectual concerns	OK	Required refactoring	OK
Trade-offs preserved	Response Time and Authenticity, Response Time and Integrity, Response Time and Encryption	Monitoring and Transaction Management, Response Time and Transaction Management, Response Time and Authenticity	Response Time and Integrity, Response Time and Encryption

Table 3
Definition of the component and concern coupling and cohesion metrics [92].

Attribute	Metric	Definition
Concern Diffusion	CDAC: Concern Diffusion over Architectural Components	It counts the number of components which contribute to the realization of a certain concern
	CDAI: Concern Diffusion over Architectural Interfaces	It counts the number of interfaces which contribute to the realization of a certain concern
Concern Coupling	CIBC: Component-level Interlacing between Concerns	It counts the number of other concerns with which the assessed concerns share at least a component
	IIBC: Interface-level Interlacing Between Concerns	It counts the number of other concerns with which the assessed concerns share at least an interface
Coupling between components	AC: Afferent Coupling Between Components	It counts the number of components which require service from the assessed component
	EC: Efferent Coupling Between Components	It counts the number of components from which the assessed component requires service
Component Cohesion	LCC: Lack of Concern-based Cohesion	It counts the number of concerns addressed by the assessed component

Table 4
Results of applying the component and concern coupling and cohesion metrics.

Attribute	Metric	Toll Gate		Auction System		Online Book Store	
		AO	*Non-AO*	*AO*	*Non-AO*	*AO*	*Non-AO*
Concern Diffusion	CDAC	1.333	2.667	2.000	2.000	1.333	2.667
	CDAI	1.000	1.000	2.000	2.000	1.333	1.333
Concern Coupling	CIBC	0.333	2.000	1.000	1.000	0.000	3.000
	IIBC	0.000	0.000	0.000	0.000	0.000	0.000
Component Coupling	AC	1.7143	1.7143	1.2857	1.333	1.000	1.000
	EC	1.1428	1.7143	1.1428	1.500	0.7500	1.000
Component Cohesion	LCC	1.000	1.7143	1.2857	1.2857	1.000	1.375

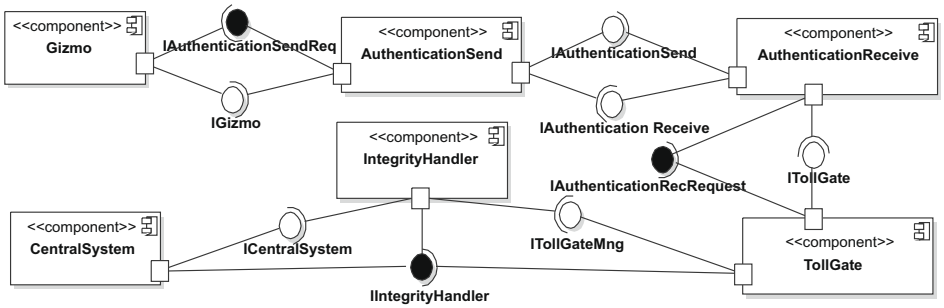

Fig. 14. Non-aspect-oriented architectural description (structural view) for the Toll Gate system.

The use of aspect-orientation removes these dependencies between base components and aspectual components (those that encapsulate crosscutting concerns). This also implies that the logics of the base components `Gizmo`, `TollGate` and `CentralSystem` have to be extended to manage these new connections. The cohesion is computed by the metric LCC (Lack of Concern-based Cohesion), in Table 3, last row. These metric counts the number of concerns addressed by the component assessed. The higher this number is, the lower the cohesion is. In the non-aspect-oriented version, base components have to deal with part of the crosscutting concerns, which contributes to increasing the average LCC, showing a lower cohesion than its aspect-oriented counterpart, where base components do not have to deal with crosscutting concerns.

Thus, a change in a newly connected component, e.g., `IntegrityHandler`, which affects its interface, would have a ripple effect on all the components connected to this interface. For instance, a change in `IntegrityHandler` that affects the interface `IIntegrityHandler` would imply changes in the components `CentralSystem` and `TollGate`. This is a consequence of the lower cohesion of the non-aspect-oriented version.

Additionally, because of these new dependencies, components are less reusable since, for instance, the component `TollGate` cannot now be reused in environments where `Integrity` or `Authenticity`, or both, are not required. The same applies to aspectual components, which have now dependencies with the base components. This is a consequence of the higher coupling of the non-aspect-oriented version.

Finally, we would like to point out that, due to the lower component coupling between components on the aspect-oriented version, the opportunities for an independent development of components increase.

7.2.2. Repetitive, laborious and error-prone tasks are automated using MDD transformations

The first step towards transforming higher abstraction model elements into lower abstraction model elements is to define and specify a set of heuristics that describe how the first elements are transformed into the second ones. These guidelines are refined until a systematic process for mapping artefacts between the two stages is obtained. Our previous experience [79,24,90,25] is to develop manual systematic processes to generate aspect-oriented architectures from aspect-oriented requirements models. Our experience shows that once the systematic process is specified, its application is a long and time-consuming task, comparable to the task of manually executing an algorithm with a large amount of input data. An additional drawback is, of course, that we cannot be sure that the end result is error-free.

Model-driven techniques enable the automation of this systematic process, allowing the definition of a set of automatic model transformations. Automation is, according to some empirical studies [10,31,81], the main contribution of model-driven development. Therefore, once we defined the process for mapping aspect-oriented requirements models into aspect-oriented architectural models, the process was encapsulated in a set of automatic model to model transformations, as described in Section 5.3.

Since these transformations are executed automatically, the obvious advantages are:

1. a significant reduction in the effort required to apply them;
2. the result is error-free, if transformations are properly developed;
3. a reduction in the level of expertise required. Since the transformations are executed automatically, engineers do not need to know exactly how they work internally. As a matter of fact, the transformations shown in this paper have been executed by our MSc students, who are not familiar with the mapping process or model transformation languages (e.g., QVT or ATL). The required knowledge about how the systematic mapping process is performed is encapsulated in the model transformation. The effort associated to execute a model transformation is reduced to the effort associated to pressing a button.

7.2.3. Model transformation helps to ensure best practices and consistency

Systematic processes for deriving architectures from requirements specifications encapsulate *best practices*. They are the result of a long and in-depth study about what the best solution is for mapping a specific problem from requirements to architecture. In our previous work [79,24,90,25], where model-driven technologies were not used, the best solution was described by means of a set of systematic or precise rules. Unlike the automatic mapping, such manual systematic processes do not ensure that engineers and developers will apply the rules correctly. We believe that automation also helps ensuring consistency in software architecture, since all the rules can be applied time and again in exactly in the same fashion.

Practical evidence of these claims can be found in [10,31,81].

7.2.4. New requirements can be more easily incorporated into the architectural model

As the systematic mapping process is automated, propagating a change from a requirements model (e.g., a new requirement addition) to an architectural model only requires the re-execution of the model transformations that generate the architectural model.

In our case studies, we experimented to add Integrity to the Online Book Store and Authenticity to the AuctionSystem. In both cases, these new changes have been successfully automatically propagated to the architectural design. Evidence of these claims can also be found in [10,31,81].

Nevertheless, if manual changes had been made on the generated architecture, these changes might be lost depending on the model transformation language used. Preservation of manual changes when a model transformation is re-executed and the

target model has been modified is still a research challenge in model transformation languages. In our case, we are using ATL, which does not provide any support for this issue. This means that the manual changes made on the software architecture will be lost, currently. We hope new features are incorporated to model transformation languages in the feature that will overcome this shortcoming.

7.2.5. Cost-benefit analysis of model-driven techniques

Constructing a model transformation implies a considerable extra initial cost, as the model transformations have to be specified, implemented, tested, debugged and finally incorporated to the production process. However, after this initial effort, the model-transformation will start to be cost-effective, due to the benefits discussed above, in particular, due to the time saved in performing manual mappings and fixing errors. Fig. 15 shows a typical graph of this cost-benefit analysis.

From our experience, the effort for specifying and implementing the model-transformations presented in this paper was quite high, mainly due to the immaturity of the tools and languages at the time of implementing them (see tool support Section 6). However, once these transformations have been implemented, they are available to be reuse for the development of other applications. Thus, the cost of the engineering effort for creating architectures from requirements remains practically unchanged as new systems are developed. If no automatic model transformations are used, this cost will increase with each system developed. However, after the *n*th system developed (see Fig. 15), the cost will be lower for the model-driven scenario. The development time decreases and, consequently, the benefits of using model-driven techniques will also grow for each system developed, from the *n*th system onwards.

Therefore, the cost-effectiveness of constructing an automatic model-transformation will depend on how reusable the transformation is. In our case, transformations of non-crosscutting requirements are metamodel-based transformations, which means they do not depend on specific elements of the model and, therefore, they can be reused for any transformation between an ASM model and a CAM model (although, as commented before, the resulting architecture may require some refactoring in certain cases). Indeed, our automatic transformations have been reused in all three case studies.

Model transformations of crosscutting requirements are pattern-based transformations that are dependent on the aspectual requirements to be transformed. Thus, a model transformation for an aspectual requirement could be reused if and only if the same aspectual requirement appears in other application model. Fortunately, most of aspectual requirements, such as Integrity, Authenticity, Encryption or Transaction are recurrent and appear repeatedly across different applications. Hence, the development of model transformations for this kind of aspectual requirement is justified. Indeed, we have reused them in our three case studies.

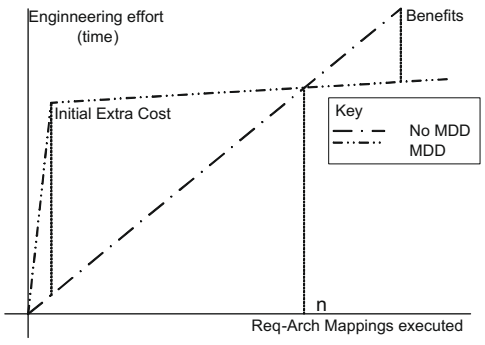

Fig. 15. Cost-effectiveness of model transformations.

7.3. Applicability, scalability and usability

This section comments on: (1) for which kind of systems our approach results more suitable and in which situations our approach may fail, i.e., it comments on threats to validity; (2) scalability issues; and (3) usability, that is, on how much effort is required for learning to use and applying our approach.

7.3.1. Applicability: threats to validity

This section evaluates under which conditions our approach is applicable and offers benefits, and under which circumstances it might fail. We have identified five scenarios where, if the conditions are not satisfied, our approach might not be suitable. We comment on these scenarios below.

7.3.1.1. Transformation for non-aspectual-requirements must be suitable. Our approach assumes that is possible to derive a base architecture for the functional requirements. The non-functional-requirements are injected as aspects into this base architecture. So, if the derivation of the base architecture is not feasible, our approach would fail.

The transformation of the non-aspectual elements is based on creating an architectural component for each lifeline contained in the scenarios modelling requirements. This transformation is suitable for those systems where domain objects are most likely to be mapped into architectural components [22]. This is the case for most of embedded systems, for example, where different devices are identified at the domain level (e.g. Toll Gate, Gizmo), and each device is mapped into an architectural component. In business applications this is not often the case. In systems like the Auction System [90], domain objects such as BankAccount are mapped into a component called VirtualBank, which is in charge of managing user back accounts. For this reason, this architecture requires some refactoring after being generated (see Table 2). Nevertheless, the transformation of non-aspectual requirements to an architectural design is not the main goal of this paper. We have addressed it only because it was needed for mapping the aspectual information. However, given the lack of work on transformations from requirements to architecture, our plan is to address this issue in the near future.

Moreover, if the generated architecture needs to be manually modified, and after that, some requirements change, manual refinements might be lost, as most of current model transformation languages do not support round-trip engineering. Nevertheless, how to deal with this issue using additional traceability mechanisms to keep track of manual changes is a hot topic in our research agenda.

7.3.1.2. The software architecture can be expressed in the UML 2.0 Profile for CAM. Our approach generates as output an aspect-oriented software architecture that is expressed in the UML 2.0 Profile for CAM. This software architecture description language has been used for expressing a wide range of systems [85,25,43,90,44], but it is unrealistic to think it can be used to express the architecture of any existing software system. So, if, for any reason, the architecture of a software system cannot be adequately expressed in the UML 2.0 Profile for CAM, our approach would be not applicable.

7.3.1.3. Aspect-orientation at the architectural level must lead to an improvement on modularization. The improvement on modularization due to the use of aspect-orientation is valid only if aspect-orientation fulfils its promise of being able to encapsulate a crosscutting concern in a single module. Whereas some works have demonstrated that concerns such as Persistence [87], Scheduling [69], Context-Awareness [101];), Distribution [97] or Coordination [40], among others [75], can be adequately modularized in a

single module using aspects, aspect-orientation should not be considered the Holy Grail in Software Engineering, as there are still some challenges to separate any kind of crosscutting concern, as discussed in [65]; Kienzle and Gélinau, 2006). So, if aspect-orientation fails on adequately encapsulating a crosscutting concern into a single module, the benefits of using aspect-oriented techniques at the architectural level would be limited, for that concern, to the automation provided to model-driven techniques, but the benefits regarding improvement on maintenance and evolution would be lost.

Nevertheless, at the requirements engineering level, the use of aspect-oriented techniques, even when aspect-orientation at the architectural design level does not make sense, can still contribute to the integration of non-functional-requirements and functional-requirements, conflict management by analysing the projection of a set of concerns on another set, and to giving to the architect more complete information about the quality attributes that are more important and need to be satisfied.

7.3.1.4. Aspect-oriented techniques must not hinder critical quality attributes. Aspect-oriented techniques improve modularization by improving coupling or cohesion. This contributes to a better evolution and maintenance. Nevertheless, aspect-oriented techniques do not promise to improve other quality factors beyond these ones [105]. Regarding other issues, such as performance, aspect-oriented techniques usually claim to be at least not worse than existing ones[15], and in other cases, such as understandability, they even acknowledge that they can be more complex than in non-aspect-oriented systems [64,80]. We believe that this is the price we must pay for an improved modularization. Thus, it is the software engineers' responsibilities to analyze the trade-offs and decide when aspect-oriented techniques contribute positively to the global goals of a project. If they find that AOSD has a negative impact, aspects should not be used and the approach presented in this paper cannot be applied – as it assumes that software engineers are interested in applying aspect-oriented technologies.

7.3.1.5. Solutions for aspectual-requirements must allow their encapsulation into a model transformation. We assume that, for each aspectual requirement: (1) a pattern solution that satisfy such an aspectual requirement exist and (2) we can encapsulate this pattern solution in a model transformation. If so, our approach brings added-value because the application of the solution pattern is automated. Nevertheless, this might be not always the case.

Let us analyze the situation when the solution to an aspectual requirement might be encapsulated into a model transformation based on a classification for aspectual requirements, as provided in Sánchez et al. [90]. Aspectual requirements are classified in Sánchez et al. [90] into *verb-based* and *adjectival-based* aspectual requirements. Verb-based aspectual requirements define some behaviour (or actions) which must be provided in order to satisfy the requirement. This behaviour crosscuts the behaviour of other requirements. Encryption and persistence are exemplars of verb-based aspectual requirements. Adjective-based aspectual requirements specify constraints or properties other requirements must fulfill. For instance, temporal constraints or requirements like: "the system must be compatible with the standard XXX"are considered adjective-based aspectual requirements.

Verb-based aspectual requirements have a strong functional part, and therefore, the solution to these aspectual requirements often imply the addition of new (aspectual) components and interfaces to the base architecture, and modify the interaction between base components. This solution can be more or less easily abstracted to a pattern, and then encapsulated into a model transformation.

For the adjective-based aspectual requirements, the scenario might be not so promising. For certain adjective-based aspectual requirements, such as compliance with standards or usability are exemplars of *adjective-based* aspectual requirements that do not have a clear architectural solution pattern and, therefore, this solution cannot be easily encapsulated in a model transformation. For other adjective-based functional requirements, such as Performance, certain solutions, such as adding a cache, might be abstracted to a pattern solution an encapsulated into an automatic model transformation.

In conclusion, we can state that our approach works well for *verb-based* aspectual requirements and, in some cases, also works for *adjective-based* aspectual requirements.

7.3.1.6. Model transformations must help to save development effort. Linking with the previous discussion, pattern solutions for verb-based aspectual requirements often imply the introduction into the base architecture a set of (aspectual) components, interfaces, new component connections and pointcuts, certain development effort is saved and our approach brings benefits.

For adjective-based aspectual requirements, this might be not always the case. The solution for certain adjective-based aspectual requirements might have a considerable functional part, such as introducing a cache. So, as in the verb-based aspectual requirements, certain effort might be saved. For other adjective-based aspectual requirements, such as ResponseTime in our case study, all that we can do is to attach constraints to the architecture to ensure that the aspectual requirement is not forgotten. Nevertheless, the effort saved is not too much and the contribution of our approach decreases.

7.3.1.7. Cost associated to the development of model transformation must be justified. As already commented, model transformation has associated a development cost that cannot be neglected. Thus, the development of model transformation for aspectual requirements only makes sense when there are opportunities for reusing such a model transformation, i.e., when the aspectual requirement and the architectural solution adopted for it are highly reusable in the development of other systems. If an aspectual requirement and/or the associated architectural solution are specific for a unique system, the development of a model transformation would be more expensive than mapping the aspectual requirements manually.

In summary, our approach is applicable to a certain system if the following conditions hold:

1. The model transformation process for the functional requirements produces a satisfactory architecture. This seems to be true for embedded and interactive systems, where each domain object is most likely to be mapped to an architectural component; however, this may not to be true for other kinds of systems, such as information systems, where a domain object is not always mapped into an architectural component.
2. The target architecture can be appropriately expressed in the UML 2.0 Profile for CAM.
3. Aspect-orientation is able to provide an adequate encapsulation for the solution of each aspectual requirements, leading to an improvement on modularization, which contributes to a better maintenance and evolution.
4. The use of aspect-orientation does not have a critical negative impact on other quality attributes of the system, such as performance or design understandability.
5. Model transformations encapsulating a satisfactory solution for the aspectual requirement exist.

[15] AOP Benchmark,http://docs.codehaus.org/display/AW/AOP+Benchmark.

6. If a satisfactory model transformation for an aspectual requirement does not exist, but a satisfactory pattern solution for such an aspectual requirement exist, we might consider to develop this model transformation and add it to the repository of available model transformation for generating aspect-oriented architectures. Nevertheless, this would only make sense if there are expectations of reusing the newly developed model transformation in several projects; otherwise, the cost associated to the development of the model transformation would not be justified.
7. The execution of these automatic model transformations must help to reduce the development effort.

7.3.2. Scalability

Regarding scalability, we did not identify critical issues in our approach, beyond the constraint of having to create a requirements model. Unfortunately, the creation of a "first model"is something we cannot skip in a model-driven development process, as we need one initial model to feed the model transformation chain. Nevertheless, the construction of this initial model can be a costly task that can hinder scalability, because there could be a noticeable gap between the aspect-oriented requirements engineering approach selected and the aspect-oriented UML scenario models needed. Thus, our recommendation goes to an aspect-oriented approach that favours the creation of scenario models, such as AORA [19,20,98,18] or aspect-oriented use cases [55].

In certain cases, it would be possible to create a model transformation to generate (semi) automatically the aspect-oriented UML scenario models. To construct this transformation, it is required that the requirements specification is conformant to a well-defined structure that can be automatically processed by a computer. This model transformation would contribute to alleviate the cost associated to the creation of the aspect-oriented UML scenario model. This task is beyond the scope of this work.

One can think that scenarios, in general, and in particular non-crosscutting sea-level scenarios (the constraint imposed on the output of the first step of our approach) hinders scalability. At the first sight, everything needs to be done "in the small"and therefore, for large and complex systems, modelling fine-grained scenarios seems like an excess of work. As explained before, the reason for that constraint is that these smaller scenarios are affected by fewer aspectual requirements, facilitating the development of automatic model transformations and the potentially required trade-off analysis between concerns. Therefore, if we look carefully, being able to define automatic transformations is a small step forward towards achieving scalability (i.e., we increase automation and reduce the manual process).

Then again, the whole system needs to be modelled and all the interactions identified, including fine-grained artefacts. That is, if we have one thousand requirements all of them need to be represented and analysed, somehow.

These two perspectives of the same problem are not contradictory and their apparent negative impact on scalability can be overcome if appropriate composition mechanisms are provided. As explained in Section 4 (step 1), large scenarios can be obtained by means of composing fine-grained ones. If details from the latter, such as event interactions, are hidden during composition, we end up with a scalable approach. This goal can be achieved, as a composition only requires the name of the fine-grained scenarios. The coarse-grained scenarios obtained through such compositions are more manageable than if their internal interactions were explicitly depicted.

Hence, if appropriate composition mechanisms and operators are provided, large scenarios can be obtained by consecutively composing simpler ones. The UML-like composition language we use provides the required mechanisms to overcome the apparent lack of scalability imposed by our initial constraint of handling only one non-crosscutting sea-level functional concern at a time.

7.3.3. Usability

This section describes the steps required for using our approach, highlighting how much effort each step requires. We assume that software development team has a large repository of model transformation encapsulating pattern solutions for aspectual requirements. Based on this assumption, the process for designing a new software architecture would be as follows.

First of all, requirements engineers create an aspect-oriented requirements engineering specification, using something such as AORA [20,98,18], Theme/Doc [8] or aspect-oriented use cases [55]. The use of these techniques should not introduce any special overhead, beyond the obvious need of learning aspect-oriented concepts if this technology is unknown to the requirements engineers. Our approach imposes the additional constraints of decomposing requirements until sea-level scenarios, but this should not introduce serious problems, as already discussed in the scalability section.

The second step is to construct a requirements model. This can be considered initially an overhead, since this step is not usually required in most development processes, and where a requirements specification is usually enough. Nevertheless, it must be taken into account that the software architecture is generated automatically, so the time required for constructing this software architecture is saved. So, if the effort required for constructing this requirement model is lower than the time required for constructing the corresponding software architecture, this overhead does not exist. Indeed, what happens is exactly the opposite, and our approach helps to reduce the development effort.

The construction of aspect-oriented requirements model does not require any special skill beyond being familiarized with aspect-orientation and UML. A wide range of UML tools can be used to build the requirements models. So, each development team can continue to use their favourite UML tool.

The third step requires the software architects to select a model transformation for each aspectual requirement. This involves navigating a repository of pattern solutions for these aspectual requirements, understanding the rationale behind each solution and, overall, understanding benefits and drawbacks of each potential solution. With this information, software architects select the model transformation that best fits each system needs. This step does not involve any different step as compared to the traditional development of software system, where software architects carry out the same steps, but using catalogues for non-functional requirements, such as [26].

Once a set of model transformations have been generated, the fourth step automatically executes the model transformations. The effort associated to this task is to navigate a list, select a set of elements and press a button. The output is an aspect-oriented software architecture.

As soon as we assume that we can reuse model transformations, there should not be serious usability issues associated to our approach. If new model transformations would be required, the scenario would change. We would need to develop new model transformation and add them to the repository of available model transformations. As already commented in the paper, the development of model transformations, overall when the target language is UML [39], might require a considerable effort, as high skills on model-driven techniques are required. Nevertheless, it is not our intention that any software architect develops model transformations. Model transformations are developed by small expert teams in model-driven techniques, following the process described Section 5.3. We would like to point out that software architects

do not need to know how these model transformations have been developed or how they work in detail. Software architects must focus on what the model transformation generates as output, rather than how the model transformations are built internally.

In summary, our approach does not have critical usability issues whenever we assume that a satisfactory repository of model transformation exists. If this repository does not exist, these model transformations should be developed by a group of experts on mode-driven techniques. After that, and if the model transformations can be reused for several projects, the cost associated to the development of the model-transformation will start to be cost-effective.

8. Related work

To the best of our knowledge this is one of the first approaches that combine MDD and AOSD to produce aspect-oriented architectures from aspect-oriented requirements models. However, there are several works combining AOSD and MDD at different developmental phases [5,72,95,12,96]. However, the focus of all these works is not on automatically deriving aspect-oriented architecture designs from aspect-oriented requirements specifications.

As stated in [3,5,72], the main motivation for applying AOSD to MDD/MDA is to solve the MDD lack of specific mechanisms for the separation of crosscutting concerns. If concerns can be modelled separately at a certain level of abstraction, the resulting models will become more manageable, extensible and maintainable.

Silaghi and Strohmeier [95] propose an approach that integrates component-based software engineering, separation of concerns, model-driven architecture, and aspect-oriented programming. All these technologies are put together in a software development method applied to enterprise, middleware-mediated applications. This work focuses on architecture specification, detailed design models and implementation, but not on how to go from requirements models to architecture models.

The proposal presented in [12] models PIM and PSM levels using Subject Oriented Design and Composition Patterns in UML. The idea is to reduce scattering and tangling in class and interaction diagrams when they realise CIM use cases. The approach models each MDA level with a set of models (ViewCIM, ViewPIM and ViewPSM) representing different aspects and viewpoints that stakeholders perceive of the software system. Therefore, the work supports the specification, modelling and transformation of large complex systems from multiple views (CIM, PIM and PSM views), but, once again, automatic transformations of requirements models to architecture models are not addressed.

Another approach, closer to ours, is the AOMDF (Aspect-Oriented Model Driven Framework) proposal [96] which proposes an MDD/MDA generation process to transform aspect-oriented models in different phases of the software life cycle. AOMDF focuses on the detailed design development phase, not addressing requirements analysis models.

Deriving aspect-oriented architectural models from an aspect-oriented requirements specification has been addressed in [25,50]. This work offers some guidelines and heuristics to support the required mappings. However, these mapping processes are manual, since they do not cope with new MDD techniques.

9. Summary and future work

This paper presents an initial step towards automating the generation of aspect-oriented architectures from aspect-oriented requirements specifications. The process discussed uses model-driven techniques and it is (semi)automatic, as the architects have to select a specific set of transformations from several possible choices, each one corresponding to a candidate architecture which satisfies the functional and the non-functional requirements. If architects do not know which software architecture best satisfies all the features and restrictions, several architecture candidates can be generated automatically with less effort. They can then apply an aspect-oriented software architecture analysis approach, for example ASAAM [102].

A case study has been used to illustrate our approach. After producing an aspect-oriented requirement specification using AORA [19,20,98,18], we identify a set of aspect-oriented scenarios, each one containing a simple (sea-level) functional requirement and a set of non-functional requirements. The resulting model is transformed into an aspect-oriented architecture model. Transformations are specified using the QVT-relational notation. Finally, an architecture for simple scenarios is obtained. This architecture satisfies functional and non-functional requirements, preserves trade-offs, and it is better modularised because crosscutting concerns are well encapsulated using aspect-oriented techniques. Non-functional requirements are satisfied, mostly using aspects, defined outside the components, which promotes parallel development and component reuse.

The approach presented in this paper was evaluated from several perspectives. On the one hand, it was applied to two other case studies – AuctionSystem (Sendal and Strohmeier, 2001) and an Online Book Store System [76] – besides the Toll Gate system used here. This confirmed that several crosscutting concerns appear once and again, and their transformations defined for one case study could be reused in the other two case studies.

On the other hand, the contribution of this paper, as well as its benefits and pitfalls, were thoroughly discussed. The specific contribution of our approach, the model-driven transformation of an aspectual requirements model into an aspect-oriented software architecture, was evaluated using the results obtained after applying the transformations to three different case studies. Additional benefits, such as the preservation of trade-offs between aspectual requirements and independence of the aspect-oriented requirements engineering approach selected were also discussed.

Benefits coming from using a combination of aspect-oriented and model-driven techniques were also assessed. A set of metrics was provided to illustrate how aspect-oriented techniques at the architectural level improve modularization, leading to a better maintenance, evolution, and reusability of the application software modules. Benefits of model-driven techniques were also discussed.

Finally, issues related to the applicability, scalability and usability of our approach were also analysed and discussed. Although the natural users of our approach are in the aspect-oriented community, we would like to encourage other software architects to use aspect-orientation, and, in particular, our approach since:

- AOSD can improve modularisation in many cases, leading to an improvement of software development, maintenance and evolution.
- AOSD promotes the reuse of architectural (and requirements analysis) artefacts. On the one hand, base artefacts are free of tangled crosscutting concerns' behaviour and, on the other hand, crosscutting concerns are encapsulated in *aspectual* modules; this makes both types of artefacts potentially more reusable.
- As the architecture obtained is better modularised, architectural constituent components will be more independent. Therefore, opportunities for developing architectural components in parallel are also increased.

As future work, we will investigate the conflicts, dependencies and interactions between scenarios, how to deal with requirements information which cannot be naturally mapped into an

architecture and more powerful techniques for trade-off analysis. Since we have also identified a lack of work on model transformations from non-aspectual requirements models to software architectural models and this is a prerequisite for a successful adoption of our work, we will address this as part of our near-future work plan.

Acknowledgements

We thank Jorge Manrique, Carlos Nebrera, Nadia Gámez and Juan Antonio Valenzuela for their help in applying and evaluating the model transformations.

This work has been partially supported by the project HP2004-0015, the Spanish Ministry Project TIN2008-01942/TIN and EC Grants IST-2-004349-NOE AOSD-Europe and AMPLE IST-033710.

References

[1] Zaid Altahat, Tzilla Elrad, Luay Tahat. Applying critical pair analysis in graph transformation systems to detect syntactic aspect interaction in UML state diagrams", in: Proceedings of the 20th International Conference on Software Engineering and Knowledge Engineering (SEKE): 905-911, Redwood City, San Francisco Bay, CA, USA, July 2008.

[2] I. Aracic, V. Gasiunas, M. Mezini, K. Ostermann, Overview of CaesarJ, in: M. Aksit, A. Rashid (Eds.), Transactions on Aspect-Oriented Software Development I (TAOSD), LNCS 3880, February 2006, pp. 135–173.

[3] M. Aksit, Systematic analysis of crosscutting concerns in the model-driven architecture design approach, Symposium on how Adaptable is MDA? Twente, Enschede, The Netherlands, May 2005.

[4] B. Alwis, G. Kiczales, Apostle: A Simple Incremental Weaver for a Dynamic Aspect Language. Technical Report TR-2003-16, Department of Computer Science, University of British Columbia, October 2003.

[5] P. Amaya, C. Gonzalez, J. Murillo, Towards a subject-oriented model-driven framework, in: Proceedings of the 1st International Workshop on Aspect-Based and Model-Based Separation of Concerns in Software Systems, AB-MB-SoC, 1st European Conference on MDA-Foundations and Applications, (ECMDA-FA), Electronic Notes on Theoretical Computer Science, Nuremberg, Germany, vol. 163(1), 2005, pp. 31–44.

[6] I. Araújo, M. Weiss, Linking patterns and non-functional requirements, Proceedings of the 9th Conference on Pattern Language of Programs, (PLoP), Monticello, IL, USA, September 2002.

[7] J. Araújo, J. Wittle, D. Kim, Modeling and composing scenario-based requirements with aspects, in: Proceedings of the 12th International Conference on Requirements Engineering (RE), Kyoto, Japan, September 2004, pp. 58–67.

[8] E. Baniassad, S. Clarke, Theme: an approach for aspect-oriented analysis and design, in: Proceedings of the 26th International Conference on Software Engineering (ICSE), Edinburgh (Scotland, United Kingdom), May 2004, pp. 158–167.

[9] E. Baniassad, P. Clements, J. Araújo, A. Moreira, A. Rashid, B. Tekinerdogan, Discovering early aspects, IEEE Software, vol. 23(1), January–February 2006, pp. 61–70.

[10] P. Baker, S. Loh, Frank Weil, Model-driven engineering in a large industrial context-motorola case study, in: Lionel C. Briand, Clay Williams (Eds.), Proceedings of the 8th International Conference on Model Driven Engineering Languages and Systems (MoDELS), LNCS 3713, Montego Bay, Jamaica, October 2005, pp. 476–491.

[11] O. Barais, E. Cariou, L. Duchien, N. Pessemier, L. Seinturier, TranSAT: a framework for the specification of software architecture evolution, in: Proceedings of the 1st International Workshop on Coordination and Adaptation Techniques (WCAT), 18th European Conference on Object-Oriented Programming (ECOOP), Oslo (Norway), June 2004.

[12] P. Barbosa, C. González, J. Murillo, MDA and separation of aspects: an approach based on multiple views and subject oriented design, in: Proceedings of the 6th International Workshop on Aspect-Oriented Modelling (AOM), 4th International Conference on Aspect-Oriented Software Development (AOSD), Chicago, IL, USA, March 2005.

[13] T. Batista, C. Chavez, A. Garcia, U. Kulesza, C. Sant'Anna, C. Lucena, Aspectual connectors: supporting the seamless integration of aspects and ADLs, in: Proceedings of the 20th Brazilian Symposium on Software Engineering (SBES), Florianopolis, Brazil, October 2006.

[14] L. Bergmans, M. Aksit, Composing crosscutting concerns using composition filters, Communications of the ACM 44 (10) (2001) 51–57.

[15] S. Beydeda, M. Book, V. Gruhn, Model-Driven Software Development, Springer, 2005.

[16] J. Bóner, A. Vasseur, AspectWerkz Web Site, <http://aspectwerkz.codehaus.org>, 2004.

[17] E. Brinksma, Information Processing Systems – Open Systems Interconnection – LOTOS: A Formal Description Technique Based on the Temporal Ordering of Observational Behaviour, ISO 8807, 1988.

[18] I. Brito, Aspect-oriented requirements analysis. Ph.D. thesis, Departamento de Informática, Faculdade de Ciências e Tecnologia, Universidade Nova de Lisboa, July 2008.

[19] I. Brito, A. Moreira, Advanced separation of concerns for requirements engineering, in: Proceedings of 8th Jornadas Ibéricas de Ingeniería del Software y Bases de Datos (JISBD), Alicante, Spain, November 2003, pp. 47–56.

[20] I. Brito, A. Moreira, Integrating the NFR framework in a RE model, in: Proceedings of the 3rd International Workshop on Early-Aspects (EA), 3rd International Conference on Aspect-Oriented Software Development (AOSD), Lancaster, England, United Kingdom, March 2004.

[21] I. Brito, F. Vieira, A. Moreira, R. ribeiro, Handling conflicts in aspectual requirements compositions, Journal of Transactions on AOSD, 2007, pp. 144–166 (special issue on Early Aspects, A. Rashid, M., Akşit (Eds.), LNCS 4620).

[22] M. Broy, K. Stølen, Specification and Development of Interactive Systems: Focus on Streams, Interfaces, and Refinement, Springer, 2001.

[23] R. Chitchyan, A. Rashid, P. Sawyer, A. Garcia, M. Pinto, J. Bakker, B. Tekinerdogan, S. Clarke, A. Jackson, Report synthesizing state-of-the-art in aspect-oriented requirements engineering, architectures and design, AOSD-Europe Deliverable D11, AOSD-Europe-ULANC-9, Lancaster University, Lancaster, England, United Kingdom, March 2005.

[24] R. Chitchyan, A. Rashid, A. Garcia, M. Pinto, P. Sánchez, L. Fuentes, A. Jackson, Mapping and refinement of requirements level aspects, AOSD-Europe Deliverable D63, AOSD-Europe-ULANC-24, Lancaster University, Lancaster, England, United Kingdom, November 2006.

[25] R. Chitchyan, M. Pinto, A. Rashid, L. Fuentes, COMPASS: composition-centric mapping of aspectual requirements to architecture, in: A. Rashid, M. Aksit (Eds.), Transactions on Aspect-Oriented Software Development IV, LNCS 4640, December 2007, pp. 3–53.

[26] L. Chung, B.A. Nixon, E. Yu, J. Mylipoulos, Non-Functional Requirements in Software Engineering, Springer, 1999.

[27] S. Clarke, Extending standard UML with model composition semantics, Science of Computer Programming 44 (1) (2002) 71–100.

[28] S. Clarke, E. Baniassad, Aspect-Oriented Analysis and Design: The Theme Approach, Addison-Wesley, 2005.

[29] A. Cockburn, Writing Effective Use Cases, Addison-Wesley, 2000.

[30] R. Coelho, A. Rashid, A. Garcia, F. Cutigi Ferrari, N. Cacho, U. Kulesza, A. von Staa, C.J. Pereira de Lucena, Assessing the impact of aspects on exception flows: an exploratory study, in: Proceedings of the 22nd European Conference on Object-Oriented Programming (ECOOP), J. Vitek, LNCS 5142, Paphos (Cyprus), July 2008, pp. 207–234.

[31] T. Cottenier, A. van den Berg, T. Elrad, The motoroal WEAVR: model weaving in a large industrial context, in: Proceedings of the 6th International Conference on Aspect-Oriented Software Development (AOSD), Industry Track, Vancouver (British Columbia, Canada), March 2006.

[32] Rémi Douence, Pascal Fradet, Mario Südholt, A framework for the detection and resolution of aspect interactions, in: Proceedings of the 1st International Conference on Generative Programming and Component Engineering (GPCE), Pittsburgh, PA, USA, October 2002, pp. 173–188.

[33] Rémi Douence, Pascal Fradet, Mario Südholt, Composition, Reuse and Interaction Analysis of Stateful Aspects, in: Proceedings of the 3rd International Conference on Aspect-Oriented Software Development (AOSD), Lancaster, United Kingdom, March 2004, pp. 141–150.

[34] Paolo Falcarin, Marco Torchiano, Automated Reasoning on Aspects Interactions, in: Proceedings of the 21st International Conference on Automated Software Engineering (ASE), Tokyo, Japan, September 2006, pp. 313–316.

[35] E. Figueiredo, N. Cacho, C. Sant'Anna, M. Monteiro, U. Kulesza, A. Garcia, S. Soares, F. Ferrari, S. Khan, F. Filho, F. Dantas, Evolving software product lines with aspects: an empirical study on design stability, in: Proceedings of the 30th International Conference on Software Engineering (ICSE), Leipzig, Germany, May 2008, pp. 261–270.

[36] E. Figueiredo, C. Sant'Anna, A. Garcia, T. Bartolomei, W. Cazzola, A. Marchetto, On the maintainability of aspect-oriented software: a concern-oriented measurement framework, in: Proceedings of the 12th European Conference on Software Maintenance and Reengineering (CSMR), Athens, Greece, April 2008.

[37] A. Finkelstein, I. Sommerville, The viewpoints FAQ, Software Engineering Journal 11 (1) (1996) 2–4 (special issue on viewpoints).

[38] R. France, I. Ray, G. Georg, S. Ghosh, An aspect-oriented approach to early design modeling, IEE Proceedings Software 151 (4) (2004) 173–185.

[39] R.B. France, S. Ghosh, T.T. Dinh-Trong, A. Solberg, Model-driven development using UML 2.0: promises and pitfalls, IEEE Computer 39 (2) (2006) 59–66.

[40] L. Fuentes, P. Sánchez, Aspect-oriented coordination, Electronic Notes in Theoretical Computer Science (ENTCS) 189 (2007) 87–103.

[41] L. Fuentes, P. Sánchez, Designing and weaving aspect-oriented executable UML models, Journal of Object Technology (JOT) 6 (7) (2007) 109–136 (special issue on Aspect-Oriented Modelling).

[42] L. Fuentes, P. Sánchez, The aspect-oriented executable UML 2.0 profile, Technical Report, University of Málaga, February 2009.

[43] L. Fuentes, M. Pinto, J.M. Troya, Supporting the development of CAM/DAOP applications: an integrated development process, Software Practice and Experience 37 (1) (2007) 21–64.

[44] L. Fuentes, M. Pinto, P. Sánchez, Generating CAM aspect-oriented architectures using model-driven development, Information of Software and Technology 50 (12) (2008) 1248–1265.

[45] A. Garcia, C. Sant'Anna, E. Figueiredo, U. Kulesza, C. Lucena, A. Staa, Modularizing design patterns with aspects: a quantitative study, in: A. Rashid, M. Aksit (Eds.), Transactions on Aspect-Oriented Software Development I, LNCS 3880, Springer, April 2006.
[46] A. Garcia, C. Chavez, T. Batista, C. Sant'Anna, U. Kulesza, A., Rashid, C.J. Pereira de Lucena, On the modular representation of architectural aspects, in: V. Gruhn, F. Oquendo (Eds.), Proceedings of the 3rd European Workshop on Software Architecture (EWSA), LNCS 4344:82-97, Nantes, France, 2006.
[47] A. Gal, W. Schröder-Preikschat, O. Spinczyk, AspectC++, Language proposal and prototype implementation, Workshop on Advanced Separation of Concerns in Object-Oriented Systems (AdSoC), 16th International Conference on Object-Oriented Programming, Systems, Languages and Applications (OOPSLA), Tampa Bay, FL, USA, October 2001.
[48] N. Gámez, Code generation from architectural descriptions based on xADL extensions, Dpto. Lenguajes y Ciencias de la Computación, Universidad de Málaga, Julio, 2007.
[49] P. Greenwood, T. Bartolomei, E. Figueiredo, M. Dósea, A. F. Garcia, N. Cacho, C. Sant'Anna, S. Soares, P. Borba, U. Kulesza, A. Rashid, On the impact of aspectual decompositions on design stability: an empirical study, in: Erik Ernst (Ed.), Proceedings of the 21st European Conference on Object-Oriented Programing (ECOOP), LNCS 4609, Berlin, Germany, July–August 2007, pp. 176–200.
[50] J. Grundy, Multi-perspective specification, design and implementation of software components using aspects, International Journal of Software Engineering and Knowledge Engineering 10 (6) (2000) 713–734.
[51] W. Harrison, H. Ossher, Subject-oriented programming, a critique of pure objects, in: Proceedings of the 8th International Conference on Object-Oriented Programming, Systems, Languages and Applications (OOPSLA), Washington, DC, USA, October 1993, pp. 411–427.
[52] J.L. Herrero, F. Sánchez, F. Lucio, M. Torro, Introducing separation of aspects at design time, Proceedings of Workshop on Aspect-Oriented Programming, in: Proceedings of the 14th European Conference on Object-Oriented Programming (ECOOP), Nantes, France, June 2000.
[53] C. Hosmer, Proving the integrity of digitals evidence with time, International Journal of Digital Evidence 1 (1) (2002).
[54] I. Jacobson, Object Oriented Software Engineering: A Use Case Driven Approach, Addison-Wesley, 1992.
[55] I. Jacobson, P.W. Ng, Aspect-Oriented Software Development with Use Cases, Addison-Wesley, 2005.
[56] Praveen K. Jayaraman, Jon Whittle, Ahmed M. Elkhodary, Hassan Gomaa, Model composition in product lines and feature interaction detection using critical pair analysis, in: Gregor Engels, Bill Opdyke, Douglas C. Schmidt, Frank Weil (Eds.), Proceedings of the 10th International Conference on Model-Driven Engineering Languages and Systems (MoDELS), LNCS 4735, Nashville, TN, USA, October 2007, pp. 151–165.
[57] F. Jouault, I. Kurtev, On the architectural alignment of ATL and QVT, in: Proceedings of the ACM Symposium on Applied Computing (SAC), Dijon, France, April 2006, pp. 1188–1195 (Chapter on Model transformation (MT)).
[58] F. Jouault, F. Allilaire, J. Bézivin, I. Kurtev, P. Valduriez, ATL: a QVT-like transformation language, in: Proceedings of the 2nd International Dynamic Languages Symposium (DLS), Companion to the 21st International Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), Portland, OR, USA, October 2006.
[59] N. Juristo, A.M. Moreno, M.I. Sanchez, Guidelines for eliciting usability functionalities, IEEE Transactions on Software Engineering 33 (11) (2007) 744–757.
[60] M. Kandé, A concern-oriented approach to software architecture. Ph.D. thesis, Swiss Federal Institute of Technology, EPFL, Lausanne, Switzerland, 2003.
[61] G. Kiczales, J. des Rivieres, D.G. Bobrow, The Art of the Metaobject Protocol, MIT Press, 1991.
[62] G. Kiczales, J. Lamping, A. Mendhekar, C. Maeda, C.V. Lopes, J.M. Loingtier, J. Irwin, Aspect-oriented programming, in: Mehmet Aksit, Satoshi Matsuoka (Eds.), Proceedings of the 11th European Conference on Object-Oriented Programming (ECOOP), LNCS 241:220-242, Jyväskylä, Finland, June 1997.
[63] G. Kiczales, E. Hilsdale, J. Hugunin, M. Kersten, J. Palm, W.G. Griswold, An overview of AspectJ, in: Jørgen Lindskov Knudsen (Ed.), Proceedings of the 15th European Conference on Object-Oriented Programming (ECOOP), LNCS 2072, Budapest, Hungary, June 2001, pp. 327–355.
[64] G. Kiczales, M. Mezini, Aspect-oriented programming and modular reasoning, in: Proceedings of the 27th International Conference on Software Engineering (ICSE), St. Louis, MO, USA, May 2005, pp. 49–58.
[65] J. Kienzle, R. Guerraoui, AOP: does it make sense? The case of concurrency and failures, in: B. Magnusson (Ed.), Proceedings of the 16th European Conference on Object-Oriented Programming (ECOOP), LNCS 2374, Málaga, Spain, June 2002, pp. 37–61.
[66] J. Kienzle, S. Gélineau, AO challenge – implementing the ACID properties for transactional objects, in: Proceedings of the 5th International Conference on Aspect-Oriented Software Development (AOSD), Bonn, Germany, March 2006, pp. 202–213.
[67] H. Kim, AspectC#: an AOSD implementation for C#, Technical Report TCD-CS-2002-55, Department of Computer Science, Trinity College, Dublin, Ireland, September 2002.
[68] A. Kleppe, J. Warmer, W. Bast, MDA Explained: The Model Driven Architecture: Practice and Promise, Addison-Wesley, 2003.
[69] K. Kourai, H. Hibino, S. Chiba, Aspect-oriented application-level scheduling for J2EE servers, in: Proceedings of the 6th International Conference on Aspect-Oriented Software Development (AOSD), Vancouver, BC, Canada, March 2007, pp. 1–13.
[70] J. Kovačević, M. Aférez, U. Kulesza, A. Moreira, J. Araújo, V. Amaral, V. Alves, A. Rashid, R. Chitchyan, Survey of the state-of-the-art in requirements engineering for software product line and model-driven requirements engineering, Deliverable D1.1, AMPLE Project, March 2007.
[71] U. Kulesza, C. Sant'Anna, A. Garcia, R. Coelho, A. von Staa, C. Lucena, Quantifying the effects of aspect-oriented programming: a maintenance study, in: Proceedings of the 9th International Conference on Software Maintenance (ICSM), Philadelphia, PA, USA, September 2006.
[72] V. Kulkarni, S. Reddy, Separation of concerns in model-driven development, IEEE Software 20 (5) (2003) 64–69.
[73] A. Lamsweerde, Goal-oriented requirements engineering: a guided tour, in: 5th International Symposium on Requirements Engineering (RE), Toronto, Ont., Canada, August 2001, pp. 249–261.
[74] K. Lieberherr, D. Orleans, J. Ovlinger, Aspect-oriented programming with adaptive methods, Communications of the ACM 44 (10) (2001) 39–41.
[75] N. Loughran, A. Rashid, R. Chitchyan, N. Leidenfrost, J. Fabry, N. Cacho, A. Garcia, F. Sanen, E. Truyen, B. De Win, W. Joosen, N. Boucké, T. Holvoet, A. Jackson, A. Nedos, N. Hatton, J. Munnelly, S. Fritsch, S. Clarke, M. Amor, L. Fuentes, M. Pinto, C. Canal, A domain analysis of key concerns – known and new candidates, AOSD-Europe Deliverable D43, AOSD-Europe-KUL-6, Katholieke Universiteit Leuven, Leuven (Belgium), February 2006.
[76] S.J. Mellor, M.J. Balcer, Executable UML: A Foundation for Model Driven Architecture, Addison-Wesley, 2002.
[77] S.J. Mellor, K. Scott, A. Uhl, D. Weise, MDA Distilled, Addison-Wesley, 2004.
[78] A. Molesini, A. Garcia, C. Chavez, T. Batista, On the quantitative analysis of architecture stability in aspectual decompositions, in: Proceedings of the 7th IEEE/IFIP Conference on Software Architecture (WICSA), Vancouver, BC, Canada, February 2008, pp. 29–38.
[79] A. Moreira, A. Rashid, J. Araújo, Multi-dimensional separation of concerns in requirements engineering, in: Proceedings of the 13th International Conference on Requirements Engineering (RE), Paris, France, August–September 2005, pp. 285–296.
[80] K. Ostermann, Reasoning about aspects with common sense, in: Proceedings of the 7th International Conference on Aspect-Oriented Software Development (AOSD), Brussels, Belgium, March–April 2008, pp. 48–59.
[81] O. Pastor, J.C. Molina, Model-Driven Architecture in Practice: A Software Production Environment Based on Conceptual Modeling, Springer, July 2007.
[82] J. Pérez, I., Ramos, J. Jaén, P. Letelier, E. Navarro, PRISMA: towards quality, aspect-oriented and dynamic software architectures, in: Proceedings of the 3rd International Conference on Quality Software (QSIC), Dallas, TX, USA, November 2003, pp. 59–66.
[83] N. Pessemier, L. Seinturier, T. Coupaye, L. Duchien, A model for developing component-based and aspect-oriented systems, in: W. Löwe, M. Süholt (Eds.), Proceedings of the 5th International Symposium on Software Composition (SC), LNCS 4089, Vienna, Austria, March 2006, pp. 259–274.
[84] M. Pinto, L. Fuentes, J.M. Troya, DAOP-ADL: an architecture description language for dynamic component and aspect-based development, in: F. Pfenning, Y. Smaragdakis (Eds.), Proceedings of the 2nd International Conference on Generative Programming and Component Engineering (GPCE), LNCS 2830, September 2003, pp. 118–137.
[85] M. Pinto, L. Fuentes, J.M. Troya, A dynamic component and aspect-oriented platform, Computer Journal 48 (4) (2005) 401–420.
[86] Object Management Group (OMG), MOF QVT Final Adopted Specification, ptc/05-11-01, November 2005.
[87] A. Rashid, A. Moreira, J. Araújo, Modularisation and composition of aspectual requirements, in: Proceedings of the 2nd International Conference on Aspect-Oriented Software Development (AOSD), Boston, MA, USA, March 2003, pp. 11–20.
[88] A. Rashid, A. Moreira, B. Tekinerdogan, Editorial: early aspects: aspect-oriented requirements engineering and architecture design, IEE Proceedings – Software 151 (4) (2004) 153–156.
[89] A. Rashid, A. Moreira, Domain models are NOT aspect free, in: O. Nierstrasz, J. Whittle, D. Harel, G. Reggio (Eds.), Proceedings of the 9th International Conference on Model Driven Engineering, Languages and Systems (MoDELS), LNCS 4199, Genova, Italy, October 2006, pp. 155–169.
[90] P. Sánchez, L. Fuentes, A. Jackson, S. Clarke, Aspects at the right time, in: A. Rashid, M. Akşit (Eds.), Transactions on Aspect-Oriented Software Development (TAOSD) IV, LNCS 4640, December 2007, pp. 54–113.
[91] F. Sanen, E. Truyen, W. Joosen, A. Jackson, A. Nedos, S. Clarke, N. Loughran, A. Rashid, Classifying and documenting aspect interactions, in: Proceedings of the 5th International Workshop on Aspects, Components, and Patterns for Infrastructure Software (ACP4IS), 5th International Conference on Aspect-Oriented Software Development (AOSD), Bonn, Germany, March 2006.
[92] C. Sant'Anna, E. Figueiredo, A.F. Garcia, C.J. Pereira de Lucena, On the modularity of software architectures: a concern-driven measurement framework, in: F. Oquendo (Ed.), Proceedings of the 1st European Conference on Software Architecture (ECSA), LNCS 4758, Aranjuez, Spain, September 2007, pp. 207–224.
[93] B. Selic, The pragmatics of model-driven development, IEEE Software 20 (5) (2003) 19–25.
[94] S. Sendall, A. Strohmeier, Specifying concurrent system behavior and timing constraints using OCL and UML, in: M. Gogolla, C. Kobryn (Eds.), Proceedings

of the 4th International Conference on the Unified Modeling Language (UML), LNCS 2185:391-405, Toronto, Canada, October 2001.
[95] R. Silaghi, A. Strohmeier, Integrating CBSE, SoC, MDA, and AOP in a software development method, in: Proceedings of the 7th Enterprise Distributed Object Computing Conference (EDOC), Brisbane, Australia, September 2003, pp. 136–146.
[96] D. Simmonds, A. Solberg, R. Reddy, R. France, S. Ghosh, An aspect oriented model driven framework, in: Proceedings of the 9th Enterprise Distributed Object Computing Conference (EDOC), Enschede, The Netherlands, September 2005, pp. 119–130.
[97] S. Soares, P. Borba, E. Laureano, Distribution and persistence as aspects, Software Practice and Experience 36 (7) (2006) 711–759.
[98] E. Soeiro, I. Brito, A. Moreira, An XML-based language for specification and composition of aspectual concerns, in: Proceedings of the 8th International Conference on Enterprise Information Systems (ICEIS), Paphos (Cyprus), May 2006, pp. 410–419.
[99] D. Stein, S. Hanenberg, R. Unland, Designing aspect-oriented crosscutting in UML, in: Proceedings of the 1st Workshop on Aspect-Oriented Modeling with the UML (AOM), 1st International Conference on Aspect-Oriented Software Development (AOSD), Enschede, The Netherlands, April 2002.
[100] P. Tarr, H. Ossher, S.M. Sutton Jr., W. Harrison, N degrees of separation: multi-dimensional separation of concerns, in: R. Filman, T. Elrad, S. Clarke, M. Aksit (Eds.), Aspect-Oriented Software Development, Capítulo 3, Addison-Wesley, October 2004, pp. 37–61.
[101] É. Tanter, K. Gybels, M. Denker, A. Bergel, Context-aware aspects, in: W. Löwe, M. Südholt (Eds.), Proceedings of the 5th International Symposium on Software Composition (SC), LNCS 4089, Vienna, Austria, March 2006, pp. 227–242.
[102] B. Tekinerdogan, ASAAM: aspectual software architecture analysis method, in: Proceedings of the 4th Working IEEE/IFIP Conference on Software Architecture (WICSA), Oslo, Norway, June 2004, pp. 5–14.
[103] J.A. Valenzuela, L.M. Fernández, M. Pinto, L. Fuentes, Deriving AO software architectures using the AO-ADL tool suite, Research Demo at the 7th International Conference on Aspect-Oriented Software Development (AOSD), Brussels, Belgium, April 2008.
[104] S. Vanstone, P. van Oorschot, A. Menezes, Handbook of Applied Cryptography, CRC Press, 1996.
[105] C. Videira Lopes, S. Bajracharya, Assessing aspect modularizations using design structure matrix and net option value, in: A. Rashid, M. Aksit (Eds.), Transactions on Aspect-Oriented Software Development (TAOSD) I, LNCS 3880, April 2006, pp. 1–35.
[106] J. Whittle, J. Araújo, Scenario modeling with aspects, in: A. Rashid, A. Moreira, B. Tekinerdogan (Eds.), IEE Proceedings – Software, August 2004 (special issue on Early Aspects: Aspect-Oriented Requirements Engineering and Architecture Design).
[107] Y. Yu, J.C. Sampaio do Prado Leite, J. Mylopoulos, From goals to aspects: discovering aspects from requirements goal models, in: Proceedings of the 12th International Conference on Requirements Engineering (RE), Kyoto, Japan, September 2008, pp. 38–47.
[108] J. Zhang, T. Cottenier, A. van den Berg, J. Gray, Aspect composition in the motorola aspect-oriented modeling weaver, Journal of Object Technology (JOT) 6 (7) (2007) 89–108.